Numerical Integration of the Polar-Factor Gradient Flow

MTH321 Project 1

Team Q
Tianrui Ji, 2361367 (Project Manager; Algorithm Implementation)
Meichen Ding, 2362344 (Mathematical Theory) Jingyan Wei, 2361411 (Mathematical Theory)
Han Liu, 2364393 (Visualization & Report) Jiasheng Xu, 2251640 (Testing & Validation)
Department of Applied Mathematics

October 11, 2026

Abstract

This project studies the numerical integration of the polar-factor gradient flow

$$\dot{\mathbf{X}} = \mathbf{X}\left(\mathbf{I}_n - \mathbf{X}^T\mathbf{X}\right),$$

which drives a nonsingular matrix toward the orthogonal factor of its polar decomposition. Three baseline numerical methods are investigated: Explicit Euler, classical fourth-order Runge–Kutta (RK4), and Implicit Euler solved by a damped Newton iteration.

The numerical solutions are validated against the exact finite-time SVD solution and an independent tight-tolerance Radau reference. The experiments examine convergence order, absolute stability, adaptive step-size control, computational work, Lyapunov decay, singular-value evolution, orthogonality defect, and rank-deficient behaviour. The measured convergence slopes are approximately 1.010, 3.927, and 1.014 for Explicit Euler, RK4, and Implicit Euler, respectively, in agreement with their theoretical orders. The exact SVD solution and the Radau reference differ by only 1.16×10^{-13} at the final time.

The results show that the explicit methods are subject to restrictive local stability limits during the rapid initial transient, whereas Implicit Euler remains more robust for coarse step sizes at the cost of nonlinear solves. Adaptive step-size control reduces the step size in the fast transient region. Under matched terminal accuracy, RK4 requires the fewest right-hand-side evaluations among the tested configurations. The rank-deficient experiment also confirms that a zero singular mode is preserved and the solution approaches a partial isometry rather than an orthogonal matrix. As an advanced comparison, a two-stage Gauss implicit Runge–Kutta method shows fourth-order behaviour and A-stability, but requires more right-hand-side evaluations than classical RK4 at the matched accuracy tested here.

Keywords: polar-factor gradient flow, Explicit Euler, Runge–Kutta method, Implicit Euler, absolute stability, adaptive time stepping

Contents

1 Introduction and Motivation **3**
1.1 Problem Background 3
1.2 Mathematical Model 3
1.3 Overview of the Report 3

2 Discretisation and Analysis **4**
2.1 Discrete Schemes . . . 4
2.1.1 Explicit Euler Method . . . 4
2.1.2 Classical Fourth-Order Runge–Kutta Method (RK4) . . . 5
2.1.3 Implicit Euler Method with Damped Newton Iteration . . . 5
2.2 Consistency and Truncation Error . . . 7
2.2.1 Theoretical Assumptions and Local Defect . . . 7
2.2.2 Derivation of Local Truncation Errors for Implemented Solvers . . . 8
2.2.3 Adaptive Step-Size Control via Step-Doubling Error Estimation . . . 10
2.2.4 Error Normalization and Step-Update Controller . . . 11
2.3 Stability Analysis . . . 11
2.3.1 Scalar Test Problem and Amplification Factors . . . 12
2.3.2 A-Stability and L-Stability Properties . . . 13
2.3.3 Problem-Specific Linearisation and Eigenvalue Spectrum (Topic 5) . . . 15
2.3.4 Theoretical Step-Size Limits vs. Empirical Disclaimers . . . 16
2.4 Convergence . . . 17
2.4.1 Discrete Error Recurrence and the Discrete Grönwall Lemma . . . 17
2.4.2 Global Convergence Order . . . 18
2.4.3 Implications for Empirical Log-Log Validation . . . 19

3 Implementation **20**
3.1 Algorithm . . . 20
3.2 Code Structure . . . 21
3.3 Output Layout and Reproduction . . . 21
3.4 Key Design Decisions . . . 21

4 Numerical Experiments **22**
4.1 Verification Against the Exact Matrix . . . 22
4.2 Stability Experiments . . . 22
4.3 Adaptivity . . . 22
4.4 Main Results and Comparison of Methods . . . 22
4.5 Verification Coverage . . . 27
4.6 Limitations and Robustness . . . 27

5 Advanced Method — Fourth-Order Gauss Implicit Runge–Kutta **27**
5.1 Method Formulation . . . 27
5.2 Solution of the Implicit Stages . . . 28
5.3 Order and Convergence . . . 28
5.4 Absolute Stability . . . 29
5.5 Numerical Comparison . . . 29

6 Conclusions **32**

References **33**

Appendices **34**

A AI Transparency Log **34**

B Individual Contribution Statements **35**

C Supplementary Figures and Data **40**
C.1 Reproduction and Evidence Files . . . 40
C.2 Data-to-Figure Map . . . 40
C.3 CSV Field Definitions and Missing Values . . . 40
C.4 Selected Numerical Evidence . . . 41
C.5 JSON Contents and Version Traceability . . . 42

1 Introduction and Motivation

1.1 Problem Background

For a nonsingular matrix $\mathbf{X}_0$, the polar decomposition writes

$$\mathbf{X}_0 = \mathbf{Q}\mathbf{H},$$

where $\mathbf{Q}$ is orthogonal and $\mathbf{H}$ is symmetric positive definite. The computation of the polar decomposition and, in particular, its orthogonal factor is a classical problem in numerical linear algebra [2].

In this project, we study a nonlinear matrix differential equation that continuously transforms a nonsingular matrix toward this orthogonal polar factor. The problem therefore combines a matrix-valued dynamical system with the numerical solution of an ordinary differential equation. Standard numerical methods for initial-value problems provide the general framework for approximating such continuous-time systems [1].

This problem is particularly suitable for numerical analysis because it has an exact finite-time solution obtained from the singular value decomposition of the initial matrix. The exact solution provides a reference against which numerical approximations can be checked.

At the same time, the model exhibits nontrivial numerical behaviour. Large initial singular values can generate a rapid transient and impose restrictive step sizes on explicit schemes. It therefore provides a useful setting for studying accuracy, absolute stability, adaptive step-size control, and computational work.

1.2 Mathematical Model

The model considered in this report is the matrix differential equation

$$\dot{\mathbf{X}}(t) = \mathbf{X}(t)\left(\mathbf{I}_n - \mathbf{X}(t)^T\mathbf{X}(t)\right), \qquad \mathbf{X}(0) = \mathbf{X}_0. \tag{1}$$

For a nonsingular initial matrix, the flow preserves the singular vectors while driving the singular values toward one. As a result, $\mathbf{X}(t)$ approaches the orthogonal polar factor as $t \to \infty$.

The system can also be interpreted as a gradient flow. Defining

$$\Phi(\mathbf{X}) = \frac{1}{4}\left\|\mathbf{X}^T\mathbf{X} - \mathbf{I}_n\right\|_F^2, \tag{2}$$

the evolution satisfies

$$\frac{d}{dt}\Phi(\mathbf{X}(t)) \leq 0.$$

Therefore, the Lyapunov functional decreases along the exact trajectory. This provides a useful structural diagnostic in addition to ordinary solution error.

1.3 Overview of the Report

In Section 2, Explicit Euler, classical fourth-order Runge–Kutta (RK4), and Implicit Euler are derived, and their consistency, stability, and convergence properties are analysed. Section 3 describes the implementation and algorithmic design. Section 4 presents the numerical experiments, including convergence, stability, adaptive step-size control, computational work, geometric diagnostics, and rank-deficient behaviour. Section 5 introduces the two-stage Gauss implicit Runge–Kutta method and compares it with the three baseline methods. Section 6 summarises the findings, limitations, and directions for future work.

2 Discretisation and Analysis

2.1 Discrete Schemes

We consider a general initial value problem (IVP), which may be non-autonomous, on the compact temporal interval $t \in [t_0, T]$:

$$\dot{\mathbf{y}}(t) = \mathbf{f}(t, \mathbf{y}(t)), \quad \mathbf{y}(t_0) = \mathbf{y}_0 \in \mathbb{R}^d, \tag{3}$$

where $\mathbf{f} : [t_0, T] \times \mathbb{R}^d \to \mathbb{R}^d$ is assumed to be sufficiently smooth and locally Lipschitz continuous with respect to $\mathbf{y}$. Let the continuous interval $[t_0, T]$ be partitioned by a uniform grid $t_n = t_0 + nh$, $n = 0, 1, \ldots, N$, where $h = (T - t_0)/N > 0$. Integrating (3) over a single time step $[t_n, t_{n+1}]$ yields the exact Volterra integral relation:

$$\mathbf{y}(t_{n+1}) - \mathbf{y}(t_n) = \int_{t_n}^{t_{n+1}} \mathbf{f}(t, \mathbf{y}(t)) \, \mathrm{d}t. \tag{4}$$

Different numerical approximations of the quadrature in (4) give rise to distinct numerical schemes with contrasting orders of accuracy, computational costs, and stability domains.

For Topic 5, the state is a square matrix $\mathbf{X}(t) \in \mathbb{R}^{n \times n}$ satisfying the autonomous matrix differential equation

$$\dot{\mathbf{X}}(t) = \mathbf{f}(\mathbf{X}(t)) = \mathbf{X}(t) \left(\mathbf{I}_n - \mathbf{X}(t)^T \mathbf{X}(t) \right), \qquad \mathbf{X}(0) = \mathbf{X}_0. \tag{5}$$

When vectorised in column-major order, $\mathbf{y} = \mathrm{vec}(\mathbf{X}) \in \mathbb{R}^{n^2}$, this becomes the standard vector-valued IVP (3); the matrix formulas below are therefore equivalent to the vector formulation. The polar-factor interpretation follows the convention in [2]. This evolution is the ambient Euclidean negative-gradient flow of $\Phi(\mathbf{X}) = \|\mathbf{X}^T\mathbf{X} - \mathbf{I}_n\|_F^2/4$. For a separate differentiable objective g constrained to the orthogonal group, let $\mathbf{G} = \nabla g(\mathbf{X})$ and $\mathbf{X}^T\mathbf{X} = \mathbf{I}_n$. Its tangent projection is

$$P_{\mathbf{X}}(\mathbf{G}) = \mathbf{G} - \mathbf{X}\,\mathrm{sym}(\mathbf{X}^T\mathbf{G}) = \mathbf{X}\,\mathrm{skew}(\mathbf{X}^T\mathbf{G}), \tag{6}$$

where $\mathrm{sym}(A) = (A + A^T)/2$ and $\mathrm{skew}(A) = (A - A^T)/2$; the corresponding constrained gradient flow is $\dot{\mathbf{X}} = -P_{\mathbf{X}}(\mathbf{G})$. The expression $(\mathbf{I}_n - \mathbf{X}\mathbf{X}^T)\mathbf{G}$ vanishes for every square orthogonal $\mathbf{X}$ and is not this tangent projection. This distinction is relevant because every orthogonal matrix is an equilibrium of the ambient flow (5).

2.1.1 Explicit Euler Method

The simplest one-step method is obtained by approximating the integral in (4) via the left-hand rectangle quadrature rule, evaluating the integrand solely at the known base state $(t_n, \mathbf{y}(t_n))$:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{f}(t_n, \mathbf{y}_n). \tag{7}$$

Equation (7) is strictly explicit; advancing the state from t_n to t_{n+1} incurs exactly one evaluation of the vector field $\mathbf{f}$ without requiring any linear or nonlinear system solves. Its cost is therefore one right-hand-side evaluation per step; the floating-point cost depends on the structure of $\mathbf{f}$ (for the matrix flow considered below, the dominant matrix products cost $\mathcal{O}(n^3)$). The method is conditionally stable and imposes severe step-size restrictions when applied to stiff ODE systems.

For the Topic 5 matrix flow (5), the Explicit Euler update is

$$\mathbf{X}_{n+1} = \mathbf{X}_n + h\,\mathbf{X}_n \left(\mathbf{I}_n - \mathbf{X}_n^T\mathbf{X}_n \right). \tag{8}$$

2.1.2 Classical Fourth-Order Runge–Kutta Method (RK4)

To achieve higher-order accuracy without computing higher-order analytical derivatives of $\mathbf{f}$, the classical fourth-order Runge–Kutta scheme (RK4) evaluates the continuous vector field at four intermediate stages across $[t_n, t_{n+1}]$:

$$\mathbf{k}_1 = \mathbf{f}(t_n, \mathbf{y}_n), \tag{9}$$

$$\mathbf{k}_2 = \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{k}_1\right), \tag{10}$$

$$\mathbf{k}_3 = \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{k}_2\right), \tag{11}$$

$$\mathbf{k}_4 = \mathbf{f}(t_n + h, \mathbf{y}_n + h\mathbf{k}_3), \tag{12}$$

and synthesizes them with the classical RK4 weights:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right). \tag{13}$$

The algebraic structure of any s-stage Runge–Kutta method can be compactly encapsulated by its Butcher tableau:

$$\begin{array}{c|c} \mathbf{c} & \mathbf{A} \\ \hline & \mathbf{b}^T \end{array} \tag{14}$$

For the matrix vector field $\mathbf{f}(\mathbf{X}) = \mathbf{X}(\mathbf{I}_n - \mathbf{X}^T\mathbf{X})$, the RK4 stages are evaluated with matrix arithmetic:

$$\mathbf{K}_1 = \mathbf{f}(\mathbf{X}_n), \tag{15}$$

$$\mathbf{K}_2 = \mathbf{f}\left(\mathbf{X}_n + \frac{h}{2}\mathbf{K}_1\right), \tag{16}$$

$$\mathbf{K}_3 = \mathbf{f}\left(\mathbf{X}_n + \frac{h}{2}\mathbf{K}_2\right), \tag{17}$$

$$\mathbf{K}_4 = \mathbf{f}\left(\mathbf{X}_n + h\mathbf{K}_3\right). \tag{18}$$

The corresponding matrix update is

$$\mathbf{X}_{n+1} = \mathbf{X}_n + \frac{h}{6}\left(\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4\right). \tag{19}$$

For the classical RK4 method, the standard coefficients $\mathbf{A} \in \mathbb{R}^{4\times4}$, $\mathbf{b} \in \mathbb{R}^4$, and $\mathbf{c} \in \mathbb{R}^4$ are:

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & 1/6 & 1/3 & 1/3 & 1/6 \end{array} \tag{20}$$

Because the Runge–Kutta matrix $\mathbf{A}$ in (20) is strictly lower triangular with a vanishing main diagonal ($a_{ij} = 0$ for all $j \geq i$), each stage derivative $\mathbf{k}_i$ depends solely on the previously evaluated stage vectors $\mathbf{k}_1, \ldots, \mathbf{k}_{i-1}$, preserving an explicit computation sequence.

2.1.3 Implicit Euler Method with Damped Newton Iteration

When the physical system exhibits stiffness, explicit schemes encounter numerical blow-up unless h is restricted to excessively small time steps. The right-hand rectangle rule gives the Implicit (Backward) Euler scheme:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{f}(t_{n+1}, \mathbf{y}_{n+1}). \tag{21}$$

Because the unknown state $\mathbf{y}_{n+1}$ appears nonlinearly within the argument of $\mathbf{f}$, (21) constitutes a coupled system of d nonlinear algebraic equations at each time step.

For Topic 5, the Implicit Euler equation becomes the nonlinear matrix equation

$$\mathbf{X}_{n+1} = \mathbf{X}_n + h\,\mathbf{X}_{n+1}\left(\mathbf{I}_n - \mathbf{X}_{n+1}^T\mathbf{X}_{n+1}\right). \tag{22}$$

Let $\mathbf{w} \in \mathbb{R}^d$ represent the unknown state vector at t_{n+1}. We define the nonlinear residual mapping $\mathbf{F} : \mathbb{R}^d \to \mathbb{R}^d$ by collecting all terms on one side:

$$\mathbf{F}(\mathbf{w}) := \mathbf{w} - \mathbf{y}_n - h\mathbf{f}(t_{n+1}, \mathbf{w}) = \mathbf{0}. \tag{23}$$

To solve $\mathbf{F}(\mathbf{w}) = \mathbf{0}$ iteratively, we linearize $\mathbf{F}$ around a current trial estimate $\mathbf{w}^{(k)}$ via first-order multivariate Taylor expansion:

$$\mathbf{F}\left(\mathbf{w}^{(k)} + \boldsymbol{\delta}^{(k)}\right) \approx \mathbf{F}\left(\mathbf{w}^{(k)}\right) + \mathbf{J}_{\mathbf{F}}\left(\mathbf{w}^{(k)}\right)\boldsymbol{\delta}^{(k)} = \mathbf{0}, \tag{24}$$

where $\mathbf{J}_{\mathbf{F}}(\mathbf{w}) \in \mathbb{R}^{d\times d}$ is the Fréchet derivative (Jacobian matrix) of the residual operator $\mathbf{F}$ with respect to $\mathbf{w}$. Applying the multivariate chain rule to (23) yields:

$$\mathbf{J}_{\mathbf{F}}(\mathbf{w}) = \frac{\partial \mathbf{F}}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}}\left(\mathbf{w} - \mathbf{y}_n - h\mathbf{f}(t_{n+1}, \mathbf{w})\right) = \mathbf{I}_d - h\mathbf{J}_{\mathbf{f}}(t_{n+1}, \mathbf{w}), \tag{25}$$

where $\mathbf{I}_d$ is the $d \times d$ identity matrix and $\mathbf{J}_{\mathbf{f}}(\cdot) = \frac{\partial \mathbf{f}}{\partial \mathbf{y}} \in \mathbb{R}^{d\times d}$ is the analytical Jacobian matrix of the continuous ODE vector field.

For the Topic 5 vector field $\mathbf{f}(\mathbf{X}) = \mathbf{X}(\mathbf{I}_n - \mathbf{X}^T\mathbf{X})$, the Fréchet derivative in a matrix direction $\mathbf{H} \in \mathbb{R}^{n\times n}$ is

$$\mathcal{D}\mathbf{f}(\mathbf{X})[\mathbf{H}] = \mathbf{H}(\mathbf{I}_n - \mathbf{X}^T\mathbf{X}) - \mathbf{X}(\mathbf{H}^T\mathbf{X} + \mathbf{X}^T\mathbf{H}). \tag{26}$$

Consequently, if $\mathbf{G}(\mathbf{X})$ denotes the matrix residual of an Implicit Euler step,

$$\mathbf{G}(\mathbf{X}) = \mathbf{X} - \mathbf{X}_n - h\mathbf{f}(\mathbf{X}), \tag{27}$$

then

$$\mathcal{D}\mathbf{G}(\mathbf{X})[\mathbf{H}] = \mathbf{H} - h\,\mathcal{D}\mathbf{f}(\mathbf{X})[\mathbf{H}]. \tag{28}$$

After column-major vectorisation, this is exactly the residual Jacobian $\mathbf{I}_{n^2} - h\mathbf{J}_{\mathbf{f}}(\mathbf{X})$ used by Newton's method.

The standard full Newton direction $\boldsymbol{\delta}^{(k)}$ is determined by solving the linear algebraic system:

$$\mathbf{J}_{\mathbf{F}}\left(\mathbf{w}^{(k)}\right)\boldsymbol{\delta}^{(k)} = -\mathbf{F}\left(\mathbf{w}^{(k)}\right). \tag{29}$$

In practical stiff and highly nonlinear implementations, the initial state $\mathbf{w}^{(0)} = \mathbf{y}_n$ may fall outside the local quadratic attraction basin, causing standard full Newton iterations ($\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \boldsymbol{\delta}^{(k)}$) to suffer from severe overshoot and divergence. We therefore use a residual-decreasing backtracking line search with damping parameter $\alpha \in (0, 1]$:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \alpha\boldsymbol{\delta}^{(k)}. \tag{30}$$

The Newton residual is accepted using the scale-aware criterion $\|\mathbf{F}(\mathbf{w})\|_\infty \le \mathrm{atol}_F + \mathrm{rtol}_F\|\mathbf{w}\|_\infty$. This prevents the stopping rule from depending on the absolute size of the matrix state alone. Algorithm 1 describes the Implicit Euler one-step solve implemented in `code/solvers.py`. It returns either the selected root or a failure signal. The outer fixed-grid solver reports that failure without changing its uniform grid; the adaptive solver rejects the trial and retries from the last accepted state with half the step.

In our runs $\varepsilon_F = 10^{-12}$ and $K_{\max} = 20$. The candidate residual is evaluated at $\mathbf{w}_{\mathrm{try}}$ in *both* occurrences of the unknown, which is essential for the line search.

Algorithm 1 One Implicit Euler step with residual-decreasing damped Newton

Input: $t_n, \mathbf{y}_n, h, \mathbf{f}, \mathbf{J}_\mathbf{f}$; tolerance ε_F; maximum Newton updates $K_{\max}$.
Output: Root $\mathbf{w}$ satisfying $\|\mathbf{F}(\mathbf{w})\|_\infty \le \varepsilon_F(1 + \|\mathbf{w}\|_\infty)$, or failure.

$\mathbf{w} \leftarrow \mathbf{y}_n$ # Previous state is the initial guess.
for $k = 0, \dots, K_{\max}$ **do**
  $\mathbf{F} \leftarrow \mathbf{w} - \mathbf{y}_n - h\mathbf{f}(t_n + h, \mathbf{w})$
  if $\mathbf{F}$ is non-finite **then**
    return failure
  end if
  if $\|\mathbf{F}\|_\infty \le \varepsilon_F(1 + \|\mathbf{w}\|_\infty)$ **then**
    return $\mathbf{w}$
  end if
  if $k = K_{\max}$ **then**
    return failure
  end if
  $\mathbf{J}_F \leftarrow \mathbf{I}_d - h\mathbf{J}_\mathbf{f}(t_n + h, \mathbf{w})$
  Solve $\mathbf{J}_F\boldsymbol{\delta} = -\mathbf{F}$; if singular, **return** failure
  $\alpha \leftarrow 1$; accepted $\leftarrow$ false
  while $\alpha \ge 10^{-4}$ **do**
    $\mathbf{w}_{\text{try}} \leftarrow \mathbf{w} + \alpha\boldsymbol{\delta}$
    $\mathbf{F}_{\text{try}} \leftarrow \mathbf{w}_{\text{try}} - \mathbf{y}_n - h\mathbf{f}(t_n + h, \mathbf{w}_{\text{try}})$
    if $\mathbf{F}_{\text{try}}$ is finite and $\|\mathbf{F}_{\text{try}}\|_\infty < \|\mathbf{F}\|_\infty$ **then**
      $\mathbf{w} \leftarrow \mathbf{w}_{\text{try}}$; accepted $\leftarrow$ true
      break
    end if
    $\alpha \leftarrow \alpha/2$
  end while
  if not accepted **then**
    return failure
  end if
end for

2.2 Consistency and Truncation Error

2.2.1 Theoretical Assumptions and Local Defect

To rigorously quantify the asymptotic discretization error of the numerical integrators formulated in Section 2.1, we establish the analytical foundation under classical regularity assumptions. Let the continuous vector field $\mathbf{f} : [t_0, T] \times \mathbb{R}^d \to \mathbb{R}^d$ satisfy the following conditions:

1. **Lipschitz Continuity:** There exists a finite Lipschitz constant $L > 0$ such that on a bounded domain $\Omega \subset [t_0, T] \times \mathbb{R}^d$:

$$\|\mathbf{f}(t, \mathbf{u}) - \mathbf{f}(t, \mathbf{v})\| \le L\|\mathbf{u} - \mathbf{v}\|, \quad \forall (t, \mathbf{u}), (t, \mathbf{v}) \in \Omega. \tag{31}$$

   By the Picard–Lindelöf theorem, together with continuity in t, condition (31) guarantees local existence and uniqueness. A solution on the full interval requires that the trajectory remain in a bounded region where these assumptions continue to hold.

2. **Sufficient Smoothness:** For the explicit remainders through $\mathcal{O}(h^3)$ in the Euler expansions and $\mathcal{O}(h^6)$ in the RK4 expansion below, assume that $\mathbf{f}$ has continuous derivatives through order five on a bounded neighbourhood of the solution and that $\mathbf{y} \in C^6[t_0, T]$. The usual order bounds $\boldsymbol{\rho}^{\text{Euler}} = \mathcal{O}(h^2)$ and $\boldsymbol{\rho}^{\text{RK4}} = \mathcal{O}(h^5)$ require fewer derivatives. For Topic 5, $\mathbf{f}(\mathbf{X}) = \mathbf{X}(\mathbf{I} - \mathbf{X}^T\mathbf{X})$ is polynomial; its exact finite-time solution in (90) is smooth, including when an initial singular value is zero, so the stronger assumptions used here hold on every finite interval.

Any generic single-step integration scheme advancing from t_n to $t_{n+1} = t_n + h$ can be cast into the increment form:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\boldsymbol{\Phi}(t_n, \mathbf{y}_n, h), \tag{32}$$

where $\mathbf{\Phi}$ represents the numerical increment function. For Explicit Euler and RK4 it is explicit. For Implicit Euler, define the one-step solution map $\mathbf{\Psi}_h(t, \mathbf{u}) = \mathbf{w}$ by

$$\mathbf{w} = \mathbf{u} + h\,\mathbf{f}(t + h, \mathbf{w}), \tag{33}$$

and define the corresponding implicit increment function by

$$\mathbf{\Phi}_{\mathrm{IE}}(t, \mathbf{u}, h) = \mathbf{f}\left(t + h, \mathbf{\Psi}_h(t, \mathbf{u})\right). \tag{34}$$

Under the *localizing assumption* (i.e., assuming the numerical solution coincides exactly with the theoretical trajectory at the current base point, $\mathbf{y}_n = \mathbf{y}(t_n)$), the one-step local error $\boldsymbol{\rho}_{n+1}$ is defined as the difference between the exact endpoint and one application of the numerical step map:

$$\boldsymbol{\rho}_{n+1} := \mathbf{y}(t_{n+1}) - \left[\mathbf{y}(t_n) + h\mathbf{\Phi}(t_n, \mathbf{y}(t_n), h)\right]. \tag{35}$$

The *local truncation error* (LTE) vector per unit step, denoted by $\boldsymbol{\tau}_{n+1}$, is defined as the normalized local defect:

$$\boldsymbol{\tau}_{n+1} := \frac{\boldsymbol{\rho}_{n+1}}{h} = \frac{\mathbf{y}(t_{n+1}) - \mathbf{y}(t_n)}{h} - \mathbf{\Phi}(t_n, \mathbf{y}(t_n), h). \tag{36}$$

A numerical scheme is defined to be *consistent* of order p if its local truncation error satisfies $\|\boldsymbol{\tau}_{n+1}\| = \mathcal{O}(h^p)$ as $h \to 0$, which is equivalent to requiring $\|\boldsymbol{\rho}_{n+1}\| = \mathcal{O}(h^{p+1})$. For Implicit Euler, this map-based local error must be distinguished from the *exact-endpoint residual* obtained by inserting the exact solution into its defining algebraic equation. The two are generally different, although they have the same leading order for sufficiently small h under the regularity and invertibility conditions given below.

2.2.2 Derivation of Local Truncation Errors for Implemented Solvers

1. Explicit Euler Scheme ($p = 1$): Performing a Taylor expansion in time of the exact continuous solution $\mathbf{y}(t_{n+1})$ about t_n yields:

$$\mathbf{y}(t_{n+1}) = \mathbf{y}(t_n) + h\dot{\mathbf{y}}(t_n) + \frac{h^2}{2}\ddot{\mathbf{y}}(t_n) + \mathcal{O}(h^3). \tag{37}$$

Recalling that $\dot{\mathbf{y}}(t_n) = \mathbf{f}(t_n, \mathbf{y}(t_n))$ and the numerical increment is $\mathbf{\Phi}_{\mathrm{EE}} = \mathbf{f}(t_n, \mathbf{y}(t_n))$, substituting (37) into (35) leads to the exact cancellation of the zeroth- and first-order terms:

$$\boldsymbol{\rho}_{n+1}^{\mathrm{EE}} = \frac{h^2}{2}\ddot{\mathbf{y}}(t_n) + \mathcal{O}(h^3) \implies \boldsymbol{\tau}_{n+1}^{\mathrm{EE}} = \frac{h}{2}\ddot{\mathbf{y}}(t_n) + \mathcal{O}(h^2). \tag{38}$$

Hence, Explicit Euler exhibits an $\mathcal{O}(h^2)$ one-step defect and is first-order consistent ($p = 1$); its leading defect term is $\frac{1}{2}h^2\ddot{\mathbf{y}}(t_n)$.

2. Implicit Euler Scheme ($p = 1$): Let $\mathbf{y}_* := \mathbf{y}(t_{n+1})$ and let $\mathbf{w}_h := \mathbf{\Psi}_h(t_n, \mathbf{y}(t_n))$ denote the selected root of the Implicit Euler equation (33). The actual local one-step error is $\boldsymbol{\rho}_{n+1}^{\mathrm{IE}} = \mathbf{y}_* - \mathbf{w}_h$, and its increment function is $\mathbf{\Phi}_{\mathrm{IE}}(t_n, \mathbf{y}(t_n), h) = \mathbf{f}(t_{n+1}, \mathbf{w}_h)$, not $\mathbf{f}(t_{n+1}, \mathbf{y}_*)$. To determine its order, first define the exact-endpoint residual

$$\mathbf{r}_{n+1}^{\mathrm{IE}} := \mathbf{y}_* - \mathbf{y}(t_n) - h\mathbf{f}(t_{n+1}, \mathbf{y}_*) = \mathbf{y}_* - \mathbf{y}(t_n) - h\dot{\mathbf{y}}(t_{n+1}). \tag{39}$$

Expanding $\dot{\mathbf{y}}(t_{n+1})$ about t_n gives

$$\dot{\mathbf{y}}(t_{n+1}) = \dot{\mathbf{y}}(t_n) + h\ddot{\mathbf{y}}(t_n) + \mathcal{O}(h^2). \tag{40}$$

Multiplying (40) by h and subtracting it from (37) gives the exact-endpoint residual:

$$\mathbf{r}_{n+1}^{\mathrm{IE}} = -\frac{h^2}{2}\ddot{\mathbf{y}}(t_n) + \mathcal{O}(h^3). \tag{41}$$

Because $\mathbf{F}(\mathbf{w}_h) = \mathbf{0}$ in (23), the difference between (39) and the numerical root satisfies the exact relation

$$\mathbf{r}_{n+1}^{\mathrm{IE}} = \boldsymbol{\rho}_{n+1}^{\mathrm{IE}} - h\big[\mathbf{f}(t_{n+1}, \mathbf{y}_*) - \mathbf{f}(t_{n+1}, \mathbf{w}_h)\big]. \tag{42}$$

Assume both endpoints and the intervening segment lie in the Lipschitz neighbourhood, the selected root exists, and $hL < 1$. Then (31) and (42) give

$$(1 - hL)\|\boldsymbol{\rho}_{n+1}^{\mathrm{IE}}\| \le \|\mathbf{r}_{n+1}^{\mathrm{IE}}\|, \qquad \|\boldsymbol{\rho}_{n+1}^{\mathrm{IE}} - \mathbf{r}_{n+1}^{\mathrm{IE}}\| \le hL\|\boldsymbol{\rho}_{n+1}^{\mathrm{IE}}\|. \tag{43}$$

Since $\mathbf{r}_{n+1}^{\mathrm{IE}} = \mathcal{O}(h^2)$, it follows that $\boldsymbol{\rho}_{n+1}^{\mathrm{IE}} = \mathbf{r}_{n+1}^{\mathrm{IE}} + \mathcal{O}(h^3)$. Thus

$$\boldsymbol{\rho}_{n+1}^{\mathrm{IE}} = -\frac{h^2}{2}\ddot{\mathbf{y}}(t_n) + \mathcal{O}(h^3), \qquad \boldsymbol{\tau}_{n+1}^{\mathrm{IE}} = -\frac{h}{2}\ddot{\mathbf{y}}(t_n) + \mathcal{O}(h^2). \tag{44}$$

Implicit Euler is therefore first-order consistent. The bound is a small-step consistency argument, not a restriction on the scalar-test-equation A-stability result below. If a Newton iteration returns an approximate root $\widetilde{\mathbf{w}}_h$, its algebraic residual $\mathbf{F}(\widetilde{\mathbf{w}}_h)$ contributes a separate solver error, controlled by the Newton stopping tolerance.

3. Classical Runge–Kutta Scheme (RK4, $p = 4$): By Taylor expanding the four intermediate slope evaluations $\mathbf{k}_1, \ldots, \mathbf{k}_4$ defined in (9)–(12) in terms of the elementary differentials of $\mathbf{f}$ and comparing the resultant linear combination against the continuous Taylor series of $\mathbf{y}(t_{n+1})$ up to fifth order, all algebraic order conditions up to order 4 are identically satisfied. Consequently:

$$\boldsymbol{\rho}_{n+1}^{\mathrm{RK4}} = \boldsymbol{\psi}(t_n)h^5 + \mathcal{O}(h^6) \implies \boldsymbol{\tau}_{n+1}^{\mathrm{RK4}} = \boldsymbol{\psi}(t_n)h^4 + \mathcal{O}(h^5), \tag{45}$$

where $\boldsymbol{\psi}(t_n)$ denotes the principal error function composed of elementary differentials of order 5. Let $\mathbf{e} = (1, 1, 1, 1)^T$ and $\mathbf{C} = \mathrm{diag}(\mathbf{c})$. The order conditions through order four are

$$\sum_i b_i = 1, \tag{46}$$

$$\sum_i b_i c_i = \frac{1}{2}, \tag{47}$$

$$\sum_i b_i c_i^2 = \frac{1}{3}, \tag{48}$$

$$\sum_{i,j} b_i a_{ij} c_j = \frac{1}{6}, \tag{49}$$

$$\sum_i b_i c_i^3 = \frac{1}{4}, \tag{50}$$

$$\sum_{i,j} b_i a_{ij} c_j^2 = \frac{1}{12}, \tag{51}$$

$$\sum_{i,j} b_i c_i a_{ij} c_j = \frac{1}{8}, \tag{52}$$

$$\sum_{i,j,k} b_i a_{ij} a_{jk} c_k = \frac{1}{24}. \tag{53}$$

The classical RK4 coefficients satisfy all these conditions. Therefore the local defect is $\mathcal{O}(h^5)$ and the global method is fourth-order accurate.

2.2.3 Adaptive Step-Size Control via Step-Doubling Error Estimation

Practical simulation of multiscale or stiff dynamical systems necessitates dynamic step-size selection to maintain uniform accuracy while minimizing computational overhead. To estimate the local one-step error without relying on analytical higher derivatives, we employ the *step-doubling* (Richardson extrapolation) strategy. In the notation of Section 2.2, this estimates the unnormalised local defect $\boldsymbol{\rho}_{n+1}$ rather than the per-unit-step LTE $\boldsymbol{\tau}_{n+1} = \boldsymbol{\rho}_{n+1}/h$.

Let the base numerical integrator have order p. From a shared accepted state $(t_n, \mathbf{y}_n)$, we compute two numerical predictions at $t_{n+1} = t_n + h$. The expansions below are local expansions: for their derivation take $\mathbf{y}_n = \mathbf{y}(t_n)$; during a computation, the same estimates refer instead to the exact IVP flow starting from the accepted numerical state $\mathbf{y}_n$, so they estimate a local step error rather than accumulated global error.

For notational precision, when $\mathbf{y}_n$ is an accepted numerical state, the symbol $\mathbf{y}(t_n + h)$ in the following local expansions denotes the exact IVP flow over one step of length h starting from $\mathbf{y}_n$ at time t_n, rather than the exact solution of the original IVP started from $\mathbf{y}_0$.

1. **Coarse Solution ($\mathbf{y}_c$):** A single trial step spanning the full increment h:

$$\mathbf{y}_c = \mathbf{y}(t_n + h) - \boldsymbol{\psi}(t_n)h^{p+1} + \mathcal{O}(h^{p+2}). \tag{54}$$

2. **Fine Solution ($\mathbf{y}_f$):** Two consecutive sub-steps spanning $h/2$. Assuming the principal error coefficient $\boldsymbol{\psi}(t)$ remains locally constant across $[t_n, t_n + h]$, the defects from both half-steps superimpose linearly:

$$\mathbf{y}_{1/2} = \mathbf{y}\left(t_n + \frac{h}{2}\right) - \boldsymbol{\psi}(t_n)\left(\frac{h}{2}\right)^{p+1} + \mathcal{O}(h^{p+2}), \tag{55}$$

$$\mathbf{y}_f = \mathbf{y}(t_n + h) - 2\boldsymbol{\psi}(t_n)\left(\frac{h}{2}\right)^{p+1} + \mathcal{O}(h^{p+2}) = \mathbf{y}(t_n + h) - \frac{\boldsymbol{\psi}(t_n)h^{p+1}}{2^p} + \mathcal{O}(h^{p+2}). \tag{56}$$

Subtracting (56) from (54) eliminates the unknown exact continuous state $\mathbf{y}(t_n + h)$:

$$\mathbf{y}_c - \mathbf{y}_f = -\boldsymbol{\psi}(t_n)h^{p+1}\left(1 - \frac{1}{2^p}\right) + \mathcal{O}(h^{p+2}) = -\frac{2^p - 1}{2^p}\boldsymbol{\psi}(t_n)h^{p+1} + \mathcal{O}(h^{p+2}). \tag{57}$$

Solving (57) directly isolates the leading error term:

$$\frac{\boldsymbol{\psi}(t_n)h^{p+1}}{2^p} = \frac{\mathbf{y}_f - \mathbf{y}_c}{2^p - 1} + \mathcal{O}(h^{p+2}). \tag{58}$$

Consequently, the computable local error estimate of the more accurate fine solution $\mathbf{y}_f$ is given by:

$$\hat{\mathbf{e}}_n := \mathbf{y}(t_n + h) - \mathbf{y}_f \approx \frac{\mathbf{y}_f - \mathbf{y}_c}{2^p - 1}. \tag{59}$$

For our implemented solvers, the denominator evaluates explicitly to:

- **Implicit / Explicit Euler ($p = 1$):** $2^1 - 1 = 1 \implies \hat{\mathbf{e}}_n \approx \mathbf{y}_f - \mathbf{y}_c$.
- **Classical RK4 ($p = 4$):** $2^4 - 1 = 15 \implies \hat{\mathbf{e}}_n \approx \frac{\mathbf{y}_f - \mathbf{y}_c}{15}$.

2.2.4 Error Normalization and Step-Update Controller

To render the acceptance decision scale-invariant across state variables of disparate magnitudes, we define a component-wise tolerance vector $\mathbf{s} \in \mathbb{R}^d$ combining user-prescribed absolute (atol) and relative (rtol) tolerances:

$$s_i = \text{atol}_i + \text{rtol} \cdot \max\left(|y_{n,i}|,\, |y_{f,i}|\right), \quad i = 1, \ldots, d. \tag{60}$$

The overall scalar normalized error ratio E is evaluated using the weighted L_∞ norm:

$$E := \max_{1 \le i \le d} \left| \frac{\hat{e}_{n,i}}{s_i} \right|. \tag{61}$$

The operational decision logic is partitioned into two mutually exclusive regimes:

- **Step Acceptance ($E \le 1$):** The estimated error satisfies the permissible tolerance corridor. The time level and state are advanced using the fine solution:

$$t_{n+1} \leftarrow t_n + h, \quad \mathbf{y}_{n+1} \leftarrow \mathbf{y}_f. \tag{62}$$

  The optional Richardson-extrapolated value is $\mathbf{y}_f + (\mathbf{y}_f - \mathbf{y}_c)/(2^p - 1)$; the controller above accepts the fine solution itself. Two half-steps reduce its leading local-error coefficient by 2^{-p} relative to one full step, but do not raise the order p of the accepted base method.

- **Step Rejection ($E > 1$):** The estimated local error exceeds the target envelope. The accepted state $(t_n, \mathbf{y}_n)$ remains unchanged, and the trial step is recomputed from that state with an attenuated step size.

Under the asymptotic error relation $E(h) \propto h^{p+1}$, setting the target normalized error for the prospective step to unity ($E(h_{\text{opt}}) \approx 1$) yields the unconstrained scale factor $r = E^{-1/(p+1)}$. To prevent step-size hunting and severe limit-cycle oscillations induced by higher-order nonlinear transients, we incorporate a conservative safety margin $S \in [0.8, 0.9]$ alongside hard growth and shrinkage limiter bounds $[r_{\min}, r_{\max}]$:

$$h_{\text{new}} = h \cdot \min\left(r_{\max},\, \max\left(r_{\min},\, S \cdot E^{-\frac{1}{p+1}}\right)\right), \tag{63}$$

where typical robust parameters are set to $S = 0.9$, $r_{\min} = 0.2$, and $r_{\max} = 2.0$. If $E = 0$, we set the scale factor directly to $r_{\max}$; the next trial step is also clipped by $T - t_n$ so that the integration terminates exactly at T. Acceptance under $E \le 1$ is a decision based on an asymptotic local-error estimate; it does not certify an exact local or global error bound. Nor does it imply absolute stability for a chosen step size: the stability condition $|R(h\lambda)| \le 1$ must be checked separately for a relevant linearised mode.

2.3 Stability Analysis

$$R_{\text{Euler}}(z) = 1 + z, \qquad R_{\text{RK4}}(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}, \qquad R_{\text{impl.Euler}}(z) = \frac{1}{1 - z}. \tag{64}$$

While consistency governs the asymptotic error behavior as $h \to 0$, the numerical feasibility of an integrator at finite step sizes is governed by absolute stability. Stiff dynamical regimes impose severe step-size restrictions on explicit integrators, rendering stability, rather than accuracy, the primary bottleneck.

2.3.1 Scalar Test Problem and Amplification Factors

We consider the classical Dahlquist scalar linear test equation:

$$\dot{y}(t) = \lambda y(t), \quad y(0) = y_0, \quad \lambda \in \mathbb{C}, \quad \mathrm{Re}(\lambda) < 0, \tag{65}$$

whose exact continuous trajectory decays monotonically: $|y(t)| = |y_0| e^{\mathrm{Re}(\lambda)t} \to 0$ as $t \to \infty$. Introducing the dimensionless complex parameter $z := h\lambda \in \mathbb{C}$, applying a generic single-step numerical integrator to (65) yields the discrete geometric recurrence:

$$y_{n+1} = R(z) y_n, \qquad y_n = [R(z)]^n \, y_0, \tag{66}$$

where $R(z) : \mathbb{C} \to \mathbb{C}$ denotes the rational or polynomial stability amplification function, at points where it is defined. The discrete perturbation sequence $\{y_n\}_{n=0}^{\infty}$ remains uniformly bounded if and only if the complex amplification factor satisfies the absolute-stability criterion:

$$|R(z)| \leq 1. \tag{67}$$

The region of absolute stability in the complex plane is formally defined as:

$$\mathcal{S} := \{ z \in \mathbb{C} : R(z) \text{ is defined and } |R(z)| \leq 1 \}. \tag{68}$$

We derive the analytic form of $R(z)$ for the three implemented solvers:

1. **Explicit Euler:** Substituting $f(y) = \lambda y$ into (7) gives:

$$y_{n+1} = y_n + h\lambda y_n = (1 + z) y_n \implies R_{\mathrm{EE}}(z) = 1 + z. \tag{69}$$

Condition (67) requires $|1 + z| \leq 1$. In the complex plane $z = x + \mathrm{i}y$, this delineates a closed circular disk of unit radius centered at $(-1, 0)$:

$$(x + 1)^2 + y^2 \leq 1. \tag{70}$$

For purely negative real eigenvalues ($\lambda = -a$ with $a > 0$, so that $z = x \in \mathbb{R}$), the stability interval is strictly restricted to $x \in [-2, 0]$, dictating the classical step-size ceiling:

$$h \leq \frac{2}{|\lambda|}. \tag{71}$$

Crucially, along the imaginary axis ($x = 0, y \neq 0$), $|R_{\mathrm{EE}}(\mathrm{i}y)|^2 = 1 + y^2 > 1$ for all $y \neq 0$. Hence, Explicit Euler contains no segment of the imaginary axis and is unconditionally unstable for purely oscillatory systems.

2. **Classical Fourth-Order Runge–Kutta (RK4):** Applying RK4 stage equations (9)–(12) recursively to $\dot{y} = \lambda y$ yields the stage slopes:

$$hk_1 = z y_n, \tag{72}$$

$$hk_2 = z \left(y_n + \frac{1}{2} hk_1 \right) = \left(z + \frac{z^2}{2} \right) y_n, \tag{73}$$

$$hk_3 = z \left(y_n + \frac{1}{2} hk_2 \right) = \left(z + \frac{z^2}{2} + \frac{z^3}{4} \right) y_n, \tag{74}$$

$$hk_4 = z \left(y_n + hk_3 \right) = \left(z + z^2 + \frac{z^3}{2} + \frac{z^4}{4} \right) y_n. \tag{75}$$

Synthesizing into the final update (13) leads directly to the truncated Taylor polynomial of the continuous exponential map e^z:

$$R_{\mathrm{RK4}}(z) = 1 + z + \frac{z^2}{2!} + \frac{z^3}{3!} + \frac{z^4}{4!}. \tag{76}$$

The stability boundary $|R_{\mathrm{RK4}}(z)| = 1$ encloses an extended domain, shown in Fig. 1. Along the negative real axis, write $z = -a$ with $a \geq 0$. One must first check that the lower boundary $R_{\mathrm{RK4}}(-a) = -1$ is never encountered. Indeed,

$$24R_{\mathrm{RK4}}(-a) = (a^2 - 2a)^2 + 8\left(a - \frac{3}{2}\right)^2 + 6 > 0 \qquad (a \in \mathbb{R}). \tag{77}$$

Consequently, $|R_{\mathrm{RK4}}(-a)| \leq 1$ is equivalent to $R_{\mathrm{RK4}}(-a) \leq 1$. Setting its nonzero endpoint equal to 1 yields

$$-a + \frac{a^2}{2} - \frac{a^3}{6} + \frac{a^4}{24} = 0 \implies a\left(a^3 - 4a^2 + 12a - 24\right) = 0. \tag{78}$$

For $q(a) := a^3 - 4a^2 + 12a - 24$, its derivative is $q'(a) = 3a^2 - 8a + 12 = 3(a - \frac{4}{3})^2 + \frac{20}{3} > 0$ for every real a. Also $q(0) = -24 < 0$ and $q(3) = 3 > 0$. Thus q has exactly one positive root $a^* \approx 2.78529$; on the relevant half-line $a \geq 0$, one has $q(a) \leq 0$ exactly for $0 \leq a \leq a^*$. Together with (77), this proves $\mathcal{S}_{\mathrm{RK4}} \cap \mathbb{R} = [-a^*, 0]$. Along the imaginary axis, direct substitution gives

$$|R_{\mathrm{RK4}}(\mathrm{i}y)|^2 = \left(1 - \frac{y^2}{2} + \frac{y^4}{24}\right)^2 + \left(y - \frac{y^3}{6}\right)^2 = 1 + \frac{y^6(y^2 - 8)}{576}. \tag{79}$$

Hence $|R_{\mathrm{RK4}}(\mathrm{i}y)| \leq 1$ exactly when $|y| \leq 2\sqrt{2}$ (with equality at $y = 0$ and at the two nonzero endpoints). This describes stability of the scalar oscillatory test mode within that interval; it is not a nonlinear stability guarantee.

3. **Implicit Euler:** Evaluating at the implicit state y_{n+1} yields:

$$y_{n+1} = y_n + zy_{n+1} \implies (1 - z)y_{n+1} = y_n \implies R_{\mathrm{IE}}(z) = \frac{1}{1 - z}. \tag{80}$$

At $z \neq 1$, the stability criterion $|R_{\mathrm{IE}}(z)| \leq 1$ is equivalent to $|1 - z| \geq 1$, which defines the exterior of the open unit disk centered at $(+1, 0)$ in the complex plane:

$$(x - 1)^2 + y^2 \geq 1. \tag{81}$$

Both figures are produced by `code/experiments.py` through `code/run_all.py`. Figure 1 shows that the Euler disk intersects the real axis at -2 and 0, while RK4 reaches -2.78529 on the negative real axis and $\pm 2\sqrt{2}$ on the imaginary axis. The Implicit Euler panel contains the closed left half-plane, as required by A-stability. The overlaid fastest contracting mode at $h = 0.08$ is $h\lambda_{\mathrm{fast}} = -2.08$: it falls outside Euler's disk but inside the RK4 and Implicit Euler regions. The open-circle mode is physical growth rather than a spurious numerical instability. In Fig. 2, the fastest contracting Topic 5 mode gives $h\lambda_{\mathrm{fast}} = -1.30$ for $h = 0.05$ and -2.60 for $h = 0.10$. The latter lies outside the Euler interval but inside the RK4 interval, matching the local estimates (98)–(99).

2.3.2 A-Stability and L-Stability Properties

A numerical integration scheme is defined to be *A-stable* if its region of absolute stability contains the closed left complex half-plane:

$$\mathbb{C}^- := \{z \in \mathbb{C} : \mathrm{Re}(z) \leq 0\} \subseteq \mathcal{S}. \tag{82}$$

For Implicit Euler, let $z = x + \mathrm{i}y$ with $x \leq 0$. The squared modulus evaluates to:

$$|1 - z|^2 = (1 - x)^2 + y^2 = 1 - 2x + x^2 + y^2. \tag{83}$$

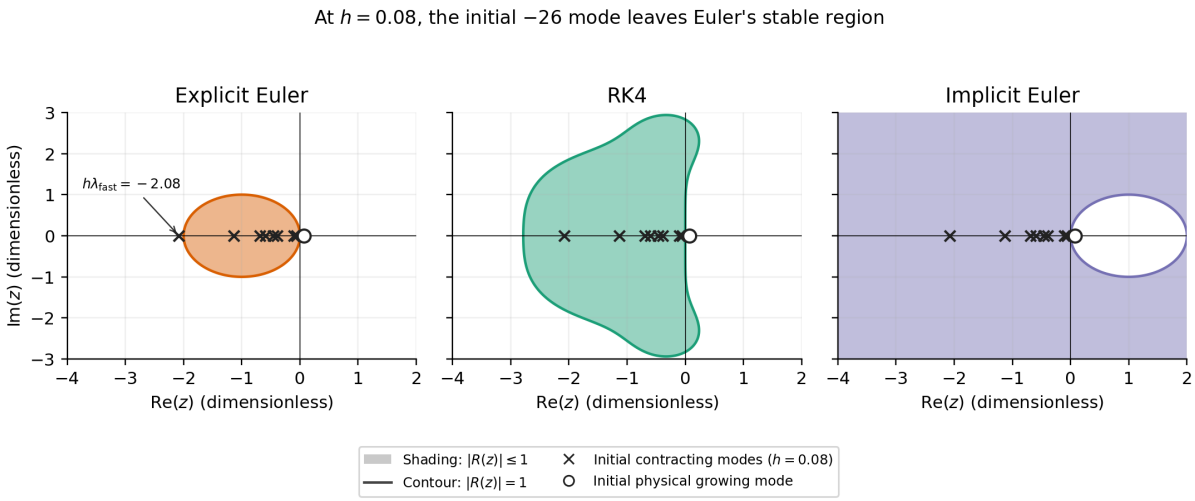


Figure 1: Absolute-stability sets $|R(z)| \leq 1$ computed from the amplification functions. The three panels show Explicit Euler, RK4, and Implicit Euler, respectively. The coloured masks visualize the analytical regions in the dimensionless $z = h\lambda$ plane. Crosses locate the eight contracting eigenmodes of the initial Jacobian at $h = 0.08$; the open circle marks the physically growing +0.88 mode. These frozen-spectrum markers do not by themselves establish nonlinear stability.

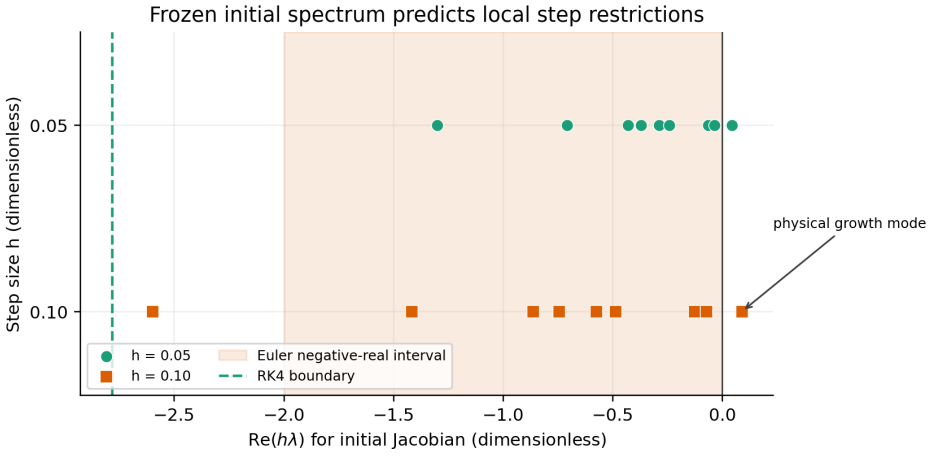


Figure 2: The initial Topic 5 Jacobian spectrum scaled by $h = 0.05$ and $h = 0.10$. The two categorical rows distinguish step sizes; their vertical positions do not represent imaginary parts. The +0.88 mode reflects physical growth of the initially small singular value, while $h = 0.10$ moves the fastest negative mode past Euler's −2 boundary.

Since $x \leq 0$, $-2x \geq 0$, and $x^2 + y^2 \geq 0$, it follows that $|1 - z|^2 \geq 1$ identically for all $z \in \mathbb{C}^-$. Therefore, Implicit Euler is unconditionally A-stable.

Furthermore, an A-stable method is classified as *L-stable* if its stability function satisfies the asymptotic damping limit at stiff infinity:

$$\lim_{|z|\to\infty} |R(z)| = 0. \tag{84}$$

Evaluating (80) gives $\lim_{|z|\to\infty} \left|\frac{1}{1-z}\right| = 0$, confirming that Implicit Euler is L-stable. For a scalar test mode with a large negative real $h\lambda$, this strongly damps the transient. Conversely, $|R_{\mathrm{EE}}(z)| \to \infty$ and $|R_{\mathrm{RK4}}(z)| \to \infty$ as $|z| \to \infty$; their polynomial stability functions therefore cannot include the entire left half-plane. These are scalar linear-test properties and do not guarantee convergence of a nonlinear Newton solve.

2.3.3 Problem-Specific Linearisation and Eigenvalue Spectrum (Topic 5)

We now specialize the stability discussion to Topic 5, the polar-factor gradient flow:

$$\dot{\mathbf{X}} = \mathbf{f}(\mathbf{X}) = \mathbf{X}\left(\mathbf{I}_n - \mathbf{X}^T\mathbf{X}\right), \qquad \mathbf{X}(0) = \mathbf{X}_0. \tag{85}$$

This is the negative gradient flow of

$$\Phi(\mathbf{X}) := \frac{1}{4}\left\|\mathbf{X}^T\mathbf{X} - \mathbf{I}_n\right\|_F^2. \tag{86}$$

Indeed, $\nabla\Phi(\mathbf{X}) = \mathbf{X}(\mathbf{X}^T\mathbf{X} - \mathbf{I}_n)$ and hence

$$\frac{\mathrm{d}}{\mathrm{d}t}\Phi(\mathbf{X}(t)) = -\left\|\mathbf{X}(t)\left(\mathbf{I}_n - \mathbf{X}(t)^T\mathbf{X}(t)\right)\right\|_F^2 \leq 0. \tag{87}$$

For the prescribed nonsingular benchmark, let

$$\mathbf{X}_0 = \mathbf{U}\,\mathrm{diag}(0.2, 1.4, 3.0)\,\mathbf{V}^T, \qquad \mathbf{U} = R(0.4), \quad \mathbf{V} = R(-0.7), \tag{88}$$

where

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

If $\mathbf{X}(t) = \mathbf{U}\boldsymbol{\Sigma}(t)\mathbf{V}^T$ with $\boldsymbol{\Sigma}(t) = \mathrm{diag}(s_1(t), \ldots, s_n(t))$, the singular vectors remain fixed and

$$\dot{s}_i(t) = s_i(t)\left(1 - s_i^2(t)\right). \tag{89}$$

Solving this scalar logistic equation gives the exact finite-time singular values

$$s_i(t) = \frac{s_{i,0}}{\sqrt{s_{i,0}^2 + \left(1 - s_{i,0}^2\right)e^{-2t}}}. \tag{90}$$

Thus $0 < s_{i,0} < 1$ gives monotone increase to 1, $s_{i,0} > 1$ gives monotone decrease to 1, and $s_{i,0} = 0$ remains zero. Scalar solutions cannot cross, so the ordering of the singular values is preserved. For a rank-deficient initial matrix, the limiting matrix is therefore a partial isometry rather than an orthogonal matrix. These facts provide exact diagnostics for the numerical experiments. As an independent sanity check, a positive diagonal initial matrix remains diagonal and each entry follows (90); this case is checked against the numerical RK4 trajectory in `test/test_validation.py`.

The Fréchet derivative in a matrix direction $\mathbf{H}$ is the operator already derived in (26). Using column-major vectorisation, $\mathrm{vec}(\mathbf{ABC}) = (\mathbf{C}^T \otimes \mathbf{A})\mathrm{vec}(\mathbf{B})$ and $\mathrm{vec}(\mathbf{H}^T) = \mathbf{K}_{n,n}\mathrm{vec}(\mathbf{H})$, this becomes

$$\mathbf{J_f}(\mathbf{X}) = \mathbf{I}_n \otimes \left(\mathbf{I}_n - \mathbf{X}\mathbf{X}^T\right) - \left(\mathbf{X}^T \otimes \mathbf{X}\right)\mathbf{K}_{n,n} - \left(\mathbf{X}^T\mathbf{X}\right)^T \otimes \mathbf{I}_n. \tag{91}$$

If a row-major vectorisation is used in code, the corresponding permutation of this Jacobian must be used instead.

For the spectral calculation write $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ and $\mathbf{H} = \mathbf{U}\mathbf{K}\mathbf{V}^T$. Then the transformed linearised operator is

$$\mathcal{L}_{\mathbf{\Sigma}}[\mathbf{K}] = \mathbf{K}(\mathbf{I}_n - \mathbf{\Sigma}^2) - \mathbf{\Sigma}(\mathbf{K}^T\mathbf{\Sigma} + \mathbf{\Sigma}\mathbf{K}). \tag{92}$$

The diagonal directions have eigenvalues

$$\lambda_{ii} = 1 - 3s_i^2. \tag{93}$$

For each pair $i < j$, the (K_{ij}, K_{ji}) block is

$$\begin{pmatrix} 1 - s_i^2 - s_j^2 & -s_i s_j \\ -s_i s_j & 1 - s_i^2 - s_j^2 \end{pmatrix}. \tag{94}$$

The eigenvector $(1,1)^T$ represents a symmetric perturbation, whereas $(1,-1)^T$ represents a skew-symmetric perturbation. Thus

$$\lambda_{ij}^{\mathrm{sym}} = 1 - s_i^2 - s_j^2 - s_i s_j, \tag{95}$$

$$\lambda_{ij}^{\mathrm{skew}} = 1 - s_i^2 - s_j^2 + s_i s_j. \tag{96}$$

At an orthogonal equilibrium $s_i = 1$, these formulas give $\lambda_{ij}^{\mathrm{sym}} = -2$ and $\lambda_{ij}^{\mathrm{skew}} = 0$. Hence symmetric perturbations are contracting normal directions, whereas skew-symmetric perturbations are neutral tangent directions along the manifold of orthogonal equilibria. The neutral tangent space has dimension $n(n-1)/2$ and the contracting normal space has dimension $n(n+1)/2$.

For the benchmark in (88), the full frozen Jacobian spectrum is

$$\sigma\left(\mathbf{J_f}(\mathbf{X}_0)\right) = \{0.88, -0.72, -1.28, -4.88, -5.76, -7.44, -8.64, -14.16, -26.0\}. \tag{97}$$

The positive eigenvalue 0.88 represents the physical growth of the singular value $s_1 = 0.2$ toward 1; it is not a numerical instability. The fastest contracting mode is -26, generated by $s_3 = 3$, while the slowest negative mode is -0.72. Thus a decay-rate spread based on the negative modes is $26/0.72 \approx 36.1$, and the transient should be described as a multiscale problem rather than as a spectrum that is entirely negative.

2.3.4 Theoretical Step-Size Limits vs. Empirical Disclaimers

At the initial state, mapping the fastest contracting spectral point $\lambda_{\mathrm{fast}} = -26.0$ onto the real stability intervals gives local frozen-Jacobian step-size estimates for the explicit solvers:

$$\text{Explicit Euler:} \quad h_{\mathrm{crit}}^{\mathrm{EE}} = \frac{2}{|\lambda_{\mathrm{fast}}|} = \frac{2}{26.0} \approx 0.0769, \tag{98}$$

$$\text{Classical RK4:} \quad h_{\mathrm{crit}}^{\mathrm{RK4}} = \frac{2.78529}{|\lambda_{\mathrm{fast}}|} = \frac{2.78529}{26.0} \approx 0.1071. \tag{99}$$

Figure 2 overlays the absolute-stability cutoffs with the scaled frozen-Jacobian eigenvalues $z_j = h\lambda_j$ at two representative step sizes. It is a local diagnostic at $\mathbf{X}_0$: because the spectrum changes along the nonlinear trajectory, it cannot replace the step-size sweeps and perturbation experiments required for a global numerical claim.

Mandatory Analytical Disclaimer: *We formally emphasize that the $h\lambda$ spectral overlay represents a local frozen-Jacobian diagnostic rather than a global nonlinear stability proof. Because the Jacobian $\mathbf{J_f}(\mathbf{X}(t))$ varies continuously along the matrix trajectory, frozen-eigenvalue containment $|R(h\lambda_j)| \leq 1$ is only a local condition and does not prove global nonlinear convergence. Finite-step nonlinear effects can still violate monotonicity or positivity properties. Consequently, the estimates* (98)–(99) *must be tested by systematic step-size sweeps and perturbation experiments.*

2.4 Convergence

Having established the consistency orders in Section 2.2, we now prove global convergence using a uniform Lipschitz bound for the one-step map and a discrete Grönwall estimate. The frozen-Jacobian stability boundaries in Section 2.3 provide separate local step-size diagnostics; they are not used as a global convergence condition.

2.4.1 Discrete Error Recurrence and the Discrete Grönwall Lemma

Let $\mathbf{y}(t) \in C^{p+1}[t_0, T]$ be the exact continuous trajectory of the initial value problem (3), and let $\{\mathbf{y}_n\}_{n=0}^{N}$ be the discrete numerical approximation generated by the one-step scheme:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\boldsymbol{\Phi}(t_n, \mathbf{y}_n, h), \quad \mathbf{y}_0 = \mathbf{y}(t_0), \tag{100}$$

where $h = (T - t_0)/N$. The true solution identically satisfies the perturbed relation:

$$\mathbf{y}(t_{n+1}) = \mathbf{y}(t_n) + h\boldsymbol{\Phi}(t_n, \mathbf{y}(t_n), h) + h\boldsymbol{\tau}_{n+1}, \tag{101}$$

where $\boldsymbol{\tau}_{n+1}$ is the local truncation error vector defined in (36). We assume that the increment function $\boldsymbol{\Phi}$ is uniformly Lipschitz in its state argument on a bounded neighborhood Ω containing the exact and numerical trajectories, with an effective Lipschitz constant $\Lambda > 0$ independent of sufficiently small h. Consequently, the one-step map $G_h(t, \mathbf{u}) = \mathbf{u} + h\boldsymbol{\Phi}(t, \mathbf{u}, h)$ has Lipschitz factor at most $1 + h\Lambda$. For Implicit Euler, this additionally requires existence of the selected nonlinear root and a uniformly bounded inverse of the residual Jacobian. More precisely, assume that for some M_J independent of sufficiently small h,

$$\sup_{\substack{(t,\mathbf{w})\in\Omega \\ t+h\leq T}} \left\| (\mathbf{I}_d - h\mathbf{J_f}(t+h, \mathbf{w}))^{-1} \right\| \leq M_J. \tag{102}$$

This condition controls the sensitivity of the selected nonlinear root to perturbations in the accepted state. We then impose the increment Lipschitz bound:

$$\|\boldsymbol{\Phi}(t, \mathbf{u}, h) - \boldsymbol{\Phi}(t, \mathbf{v}, h)\| \leq \Lambda\|\mathbf{u} - \mathbf{v}\|, \quad \forall (t, \mathbf{u}), (t, \mathbf{v}) \in \Omega, \ \forall h \in (0, h_0]. \tag{103}$$

We define the global discretization error at grid node t_n as:

$$\mathbf{e}_n := \mathbf{y}(t_n) - \mathbf{y}_n, \quad n = 0, 1, \ldots, N. \tag{104}$$

Subtracting (100) from (101) yields the dynamic recurrence for the error vector:

$$\mathbf{e}_{n+1} = \mathbf{e}_n + h\left[\boldsymbol{\Phi}(t_n, \mathbf{y}(t_n), h) - \boldsymbol{\Phi}(t_n, \mathbf{y}_n, h)\right] + h\boldsymbol{\tau}_{n+1}. \tag{105}$$

Taking Euclidean norms on both sides of (105) and applying the triangle inequality alongside (103):

$$\begin{aligned} \|\mathbf{e}_{n+1}\| &\leq \|\mathbf{e}_n\| + h\|\boldsymbol{\Phi}(t_n, \mathbf{y}(t_n), h) - \boldsymbol{\Phi}(t_n, \mathbf{y}_n, h)\| + h\|\boldsymbol{\tau}_{n+1}\| \\ &\leq \|\mathbf{e}_n\| + h\Lambda\|\mathbf{y}(t_n) - \mathbf{y}_n\| + h\|\boldsymbol{\tau}_{n+1}\| \\ &= (1 + h\Lambda)\|\mathbf{e}_n\| + h\|\boldsymbol{\tau}_{n+1}\|. \end{aligned} \tag{106}$$

Let $\tau_{\max}(h) := \max_{1\le k\le N} \|\boldsymbol{\tau}_k\|$. Unrolling the scalar recurrence (106) inductively from the initial state yields:

$$\|\mathbf{e}_n\| \le (1+h\Lambda)^n \|\mathbf{e}_0\| + h\tau_{\max}(h) \sum_{j=0}^{n-1} (1+h\Lambda)^j. \tag{107}$$

Assuming exact initial data ($\|\mathbf{e}_0\| = 0$) and summing the finite geometric progression with ratio $1 + h\Lambda > 1$:

$$\sum_{j=0}^{n-1} (1+h\Lambda)^j = \frac{(1+h\Lambda)^n - 1}{(1+h\Lambda) - 1} = \frac{(1+h\Lambda)^n - 1}{h\Lambda}. \tag{108}$$

Substituting (108) into (107), and noting the fundamental inequality $1 + x \le e^x$ for all $x \ge 0$, we have:

$$(1+h\Lambda)^n \le \left(e^{h\Lambda}\right)^n = e^{nh\Lambda} = e^{\Lambda(t_n - t_0)} \le e^{\Lambda(T-t_0)}. \tag{109}$$

Combining (107)–(109) produces the classical discrete Grönwall bound on the global discretization error (with the continuous limit understood when $\Lambda = 0$):

$$\|\mathbf{e}_n\| \le \frac{e^{\Lambda(T-t_0)} - 1}{\Lambda} \tau_{\max}(h), \quad \forall n = 0, 1, \ldots, N. \tag{110}$$

2.4.2 Global Convergence Order

The bound (110) provides the fundamental mathematical bridge between the local truncation error rate $\boldsymbol{\tau}$ and the global error $\mathbf{e}_n$:

- **Explicit / Implicit Euler ($p = 1$):** Since $\|\boldsymbol{\tau}_{n+1}\| \le C_1 h$, the terminal error satisfies:

$$\|\mathbf{e}_N\| \le \left(\frac{e^{\Lambda(T-t_0)} - 1}{\Lambda} C_1 \right) h = \mathcal{O}(h). \tag{111}$$

- **Classical Runge–Kutta (RK4, $p = 4$):** Since $\|\boldsymbol{\tau}_{n+1}\| \le C_4 h^4$, the terminal error satisfies:

$$\|\mathbf{e}_N\| \le \left(\frac{e^{\Lambda(T-t_0)} - 1}{\Lambda} C_4 \right) h^4 = \mathcal{O}(h^4). \tag{112}$$

The preceding estimates assume that the nonlinear equation in each Implicit Euler step is solved sufficiently accurately. If the Newton iteration is stopped with an algebraic residual larger than the time-discretisation error, the observed rate can be limited by the nonlinear-solver tolerance rather than by the formal order of the time-stepping method. In computations, the Newton residual tolerance must therefore be chosen below the target time-discretisation error.

More precisely, to retain the first-order global convergence result as $h \to 0$, the algebraic error introduced by each Implicit Euler solve should be of the same order as, or smaller than, the local time-discretisation defect. Thus a sufficient asymptotic requirement is that the per-step nonlinear-solver residual scale as $\mathcal{O}(h^2)$. A fixed Newton tolerance such as 10^{-12} should therefore be interpreted as adequate over the finite range of step sizes used in the reported experiments, rather than as an $h \to 0$ convergence assumption.

For the fixed-grid Implicit Euler experiments with $N = 40, 80, 160, 320$, repeating the computation with a tighter Newton tolerance of 10^{-14} changes the terminal solution by at most about 1.14×10^{-8} relative to the corresponding discretisation error. Hence the chosen 10^{-12} tolerance is negligible for the reported convergence experiment. This explains the apparent paradox regarding order indices: while an individual integration step generates a local defect of order $\mathcal{O}(h^{p+1})$, the accumulation of $N = \mathcal{O}(h^{-1})$ steps over a compact temporal interval $[t_0, T]$ causes the discrete sequence to converge globally at order $\mathcal{O}(h^p)$.

For standard one-step methods, consistency together with a uniform Lipschitz stability bound implies convergence. In the present setting this is exactly the role of the estimate above; the multistep characteristic-polynomial root condition is not needed for Explicit Euler, RK4, or Implicit Euler.

2.4.3 Implications for Empirical Log-Log Validation

For Topic 5, the finite-time exact matrix solution associated with the benchmark in (88) is

$$\mathbf{X}_{\text{exact}}(t) = \mathbf{U}\ \text{diag}\big(s_1(t), \ldots, s_n(t)\big)\, \mathbf{V}^T, \tag{113}$$

where the singular values are given by (90). The terminal numerical error is measured in the Frobenius norm:

$$E_h(T) = \|\mathbf{X}_h(T) - \mathbf{X}_{\text{exact}}(T)\|_F\,. \tag{114}$$

Let $\mathcal{E}_h = \|\mathbf{e}_N(h)\|$ for the vector formulation and $\mathcal{E}_h = E_h(T)$ for Topic 5. Taking the natural logarithm of the asymptotic relation $\mathcal{E}_h \approx Ch^p$ yields:

$$\ln \mathcal{E}_h \approx p \ln h + \ln C. \tag{115}$$

Equation (115) establishes that plotting the terminal error (using $E_h(T)$ for Topic 5) against the discretisation step size on a logarithmic scale yields an affine straight line whose asymptotic slope is the theoretical convergence order p. In the numerical-experiments section, this relation serves as the primary quantitative metric for Explicit Euler ($p \approx 1$), Implicit Euler ($p \approx 1$), and classical RK4 ($p \approx 4$). For two consecutive step sizes h and $h/2$, the observed order is estimated by

$$p_{\text{obs}}(h) = \frac{\log\big(E_h(T)/E_{h/2}(T)\big)}{\log 2}. \tag{116}$$

Thus the expected asymptotic values are $p_{\text{obs}} \approx 1$ for Explicit and Implicit Euler and $p_{\text{obs}} \approx 4$ for RK4.

3 Implementation

3.1 Algorithm

The three one-step updates are (7), (13), and (21); the last calls Algorithm 1. For a fixed-grid convergence run, the implementation chooses an integer N, sets $h = (T - t_0)/N$, and performs exactly N updates. A failed nonlinear solve raises an error so that no grid is silently changed. Algorithm 2 describes the separate adaptive controller implemented for all three base methods. A standard treatment of these one-step methods appears in [1].

Algorithm 2 Step-doubling adaptive integration

Input: Method Ψ_h of order p; $t_0, \mathbf{y}_0, T, h$; atol > 0, rtol ≥ 0.
Output: Accepted trajectory ending at T, or a reported failure.

$(t, \mathbf{y}) \leftarrow (t_0, \mathbf{y}_0)$
while $t < T$ and the trial limit is not exhausted **do**
  $h \leftarrow \min\{h, T - t\}$
  if h falls below the permitted minimum away from the final endpoint **then**
    return failure
  end if
  Try $\mathbf{y}_c = \Psi_h(t, \mathbf{y})$, $\mathbf{y}_m = \Psi_{h/2}(t, \mathbf{y})$, and $\mathbf{y}_f = \Psi_{h/2}(t + h/2, \mathbf{y}_m)$
  if a nonlinear one-step solve reports failure **then**
    Reject; $h \leftarrow h/2$; **continue** from $(t, \mathbf{y})$
  end if
  $\widehat{\mathbf{e}} \leftarrow (\mathbf{y}_f - \mathbf{y}_c)/(2^p - 1)$
  $s_i \leftarrow \text{atol} + \text{rtol} \max\{|y_i|, |y_{f,i}|\}$; $E \leftarrow \max_i |\widehat{e}_i|/s_i$
  if E is non-finite **then**
    $E \leftarrow \infty$
  end if
  if $E \leq 1$ **then**
    Accept: $(t, \mathbf{y}) \leftarrow (t + h, \mathbf{y}_f)$; store the time, state, actual h, and E
    if $t = T$ **then**
      return accepted trajectory
    end if
  else
    Reject; keep $(t, \mathbf{y})$ unchanged
  end if
  $r \leftarrow 2$ if $E = 0$; otherwise $r \leftarrow \text{clip}(0.9E^{-1/(p+1)}, 0.2, 2)$
  $h \leftarrow rh$
end while
return failure if T was not reached

3.2 Code Structure

File	Purpose
`code/model.py`	Topic 5 benchmark, column-major vectorization, matrix RHS, analytic Jacobian, exact SVD solution, and diagnostics.
`code/solvers.py`	Explicit Euler, RK4, Implicit Euler with damped Newton, fixed-grid and adaptive drivers, and work counters.
`code/experiments.py`	Convergence, reference, stability, trajectory, adaptivity, cost, and rank-deficient experiments; exports PNGs to `figures/`, and CSV/JSON to `data/`.
`code_advanced/`	Gauss IRK4 solver, four-method comparison, four additional figures, advanced data, and dedicated checks.
`code/run_all.py`	One command regenerates every report figure.
`test/`	Model, order, geometry, and adaptivity checks for all three methods.
`report_v3/data/`	Submitted baseline CSV/JSON evidence documented in Appendix C.

Table 1: Files used to reproduce the numerical results.

3.3 Output Layout and Reproduction

Run `python code/run_all.py` from `Project1/`. The source directory contains only source and documentation. The command regenerates eight baseline images in `figures/` and four advanced comparison images in `figures/advanced_irk4/`. It writes six baseline CSV files and `summary.json` to `data/`, and two advanced CSV files and a separate `summary.json` to `data/advanced_irk4/`. All paths are resolved from the entry-point file rather than a machine-specific drive or the shell's current directory.

This report version bundles all twelve figure snapshots and the baseline data snapshots. Appendix C maps the baseline evidence files to their figures, fields, and claims; Section 5 identifies the project-level advanced data files. Reproduction writes to the project-level output directories; it does not overwrite archived report versions. `python -B -m unittest discover -s test -v` runs the numerical checks for the three baseline methods. The six advanced-method checks are run from `code_advanced/test_gauss_irk4.py`. The version index in `notes/REPORT_VERSIONS.md` identifies the current report and the Git tags used to recover earlier complete project states.

3.4 Key Design Decisions

The state is a nine-component column-major vector $\mathrm{vec}(X)$ throughout. The analytic Fréchet derivative (26) supplies Newton's Jacobian; Newton starts from the previous accepted state and uses the scale-aware residual threshold $\|F(w)\|_\infty \le 10^{-12}(1 + \|w\|_\infty)$. Backtracking accepts only a finite candidate whose *complete* residual decreases. The adaptive controller accepts two half steps, not a Richardson-corrected state, so its base method retains order p. It reports accepted and rejected trials separately. The exact SVD solution is excluded from all solver steps and is used only for validation; an independently integrated Radau trajectory through SciPy provides a second check [3]. Dependencies, invocation commands, and every generated figure are documented in the project README.

As an independent check of the implicit-step Jacobian, we use the benchmark vector $y_0 = \mathrm{vec}(X_0)$, $h = 0.05$, and direction $v = (0.1, 0.2, \ldots, 0.9)^T$. With $F(w) = w - y_0 - hf(h, w)$ and $\varepsilon = 10^{-6}$, the centred difference $[F(y_0 + \varepsilon v) - F(y_0 - \varepsilon v)]/(2\varepsilon)$ differs from $(I - hJ_f(y_0))v$ by 3.68×10^{-10} in the infinity norm. This checks the complete residual derivative, including the identity term; the reproducible calculation is in `test/test_validation.py`.

4 Numerical Experiments

4.1 Verification Against the Exact Matrix

We integrate the prescribed X_0 over $[0, 2]$ on fixed uniform grids and measure the full-matrix terminal error

$$E_h(2) = \|X_h(2) - X_{\text{exact}}(2)\|_F.$$

The limiting orthogonal factor is not the finite-time reference. Euler and Implicit Euler use $N \in \{40, 80, 160, 320\}$. RK4 also uses $N \in \{640, 1280, 2560\}$. The finest three RK4 grids are used for the displayed slope because the coarse-grid ratios are affected by the rapid initial transient. The smallest measured error is near floating-point round-off scale, so the fitted fine-grid slope is a descriptive validation diagnostic; no flat round-off floor has been observed on these grids. Table 2 and Fig. 3 report the measured orders.

Method	Fitted slope	Finest h	Finest terminal error
Explicit Euler	1.010	0.00625	4.6141×10^{-4}
RK4	3.927	0.00078125	7.3968×10^{-14}
Implicit Euler	1.014	0.00625	4.6429×10^{-4}

Table 2: Finite-time full-matrix Frobenius error. The complete grid-by-grid values and adjacent observed orders are in `data/convergence.csv`.

A separate `solve_ivp` Radau run with relative tolerance 10^{-12} and absolute tolerance 10^{-14} differs from the exact SVD matrix at $t = 2$ by 1.16×10^{-13} in Frobenius norm. This cross-check and the analytic formula are independent of the three implemented time-stepping methods.

4.2 Stability Experiments

The initial frozen-Jacobian estimates $h_{\text{crit}}^{\text{EE}} \approx 0.0769$ and $h_{\text{crit}}^{\text{RK4}} \approx 0.1071$ predict the onset of possible amplification in the fastest contracting direction. We test selected fixed steps $h = 2/N$ with $N = 6, 7, 8, 10, 12, 16, 20, 24, 32, 40, 50, 64, 80$ and perturb the initial state by 10^{-7} in that direction. At $h = 0.10$, one Euler step amplifies this perturbation by about 1.60, although the full numerical path still finishes. At $h = 2/7$, Euler increases Φ by 31.9 on its first step and the run subsequently fails. RK4 at $h = 1/3$ also fails after a large first-step energy rise, whereas Implicit Euler finishes that tested grid. These observations support the predicted local risk but do not turn the frozen-eigenvalue cutoffs into nonlinear global thresholds.

4.3 Adaptivity

For all three base methods, the step-doubling trials start at $h = 0.15$ with atol $= 10^{-8}$ and rtol $= 10^{-6}$. The fine result is accepted only when its normalized estimated local error is at most one. Table 3 and Fig. 5 show that the accepted step size genuinely changes. A very short final step may be caused by clipping to reach $t = 2$ exactly. These local tolerances do not provide a strict bound on the terminal global error, which we measure separately against the exact SVD matrix.

4.4 Main Results and Comparison of Methods

Geometry and Lyapunov diagnostics. On the RK4 trajectory with $h = 0.005$, every sampled singular value moves toward one without crossing it, and $\Phi(X)$ decreases at every stored step. The numerical orthogonality defect at $t = 2$ is 0.305932824226, compared with the nonzero exact finite-time value 0.305932824210. Figure 6 also shows the finite-difference check

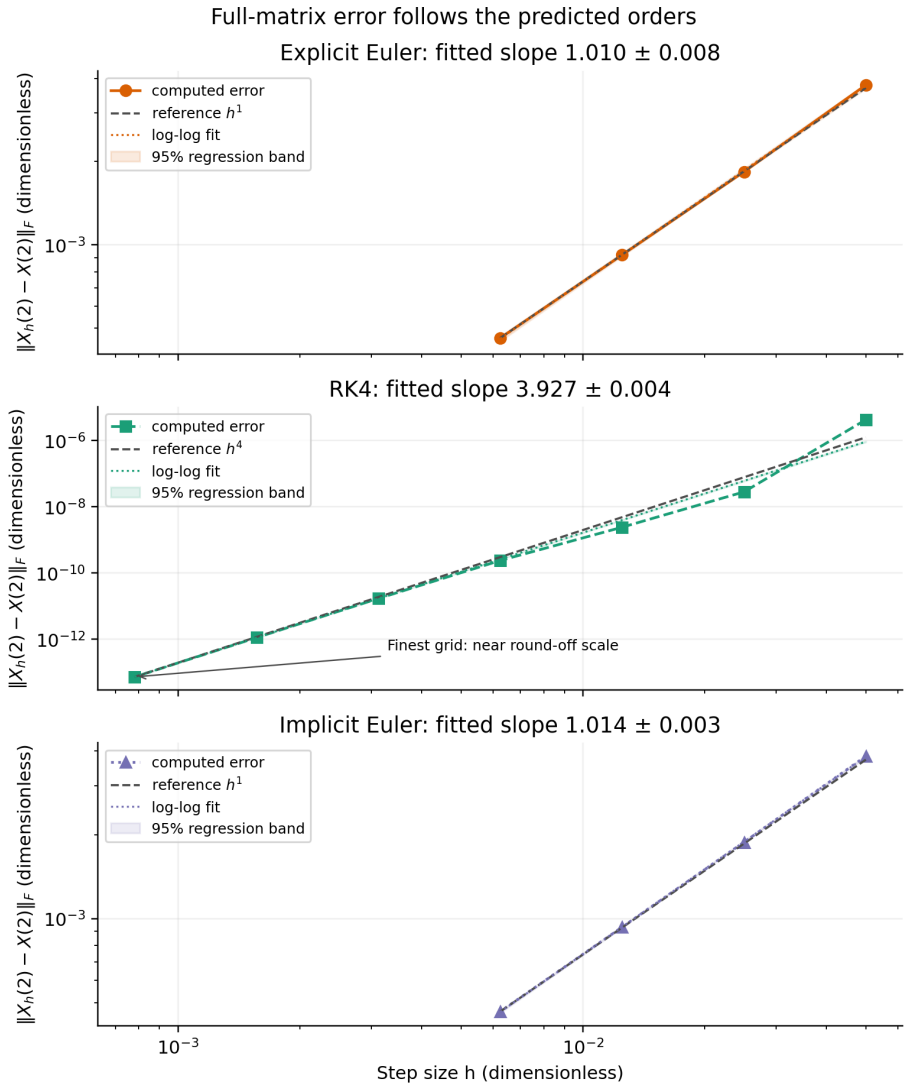


Figure 3: Terminal full-matrix error versus dimensionless step size, with h^1 or h^4 reference slopes and fitted slopes from the refined grids. The shaded 95% regression bands and reported slope standard errors are descriptive least-squares diagnostics for deterministic mesh data, not probabilistic uncertainty in the ODE solution. The agreement verifies the predicted global orders against the finite-time exact matrix.

Method	Accepted	Rejected	Max. accepted E	Terminal error
Explicit Euler	1680	5	0.842	5.08×10^{-5}
RK4	17	2	0.754	6.89×10^{-7}
Implicit Euler	1678	5	0.843	5.09×10^{-5}

Table 3: Step-doubling runs with the same local tolerance settings. The numbers of accepted steps alone are not a work comparison: each trial computes one full and two half steps, and rejected trials also consume evaluations.

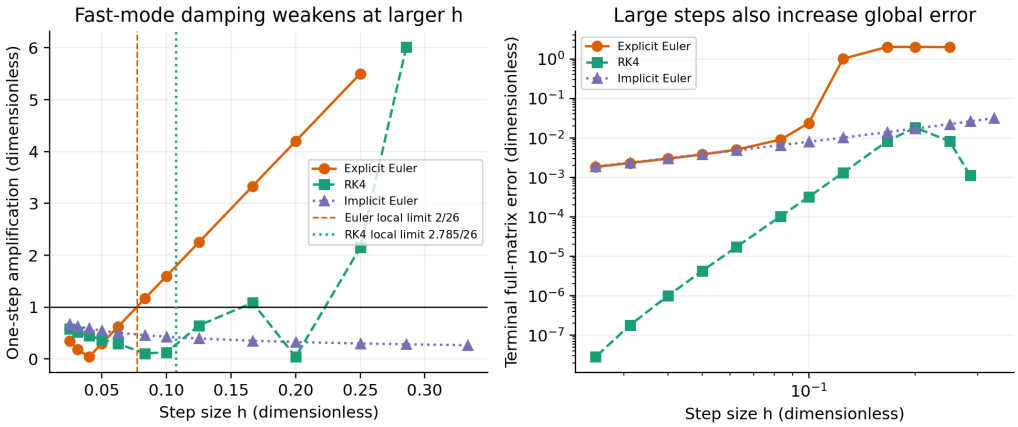


Figure 4: Left: one-step amplification of a fast-direction initial perturbation. Right: terminal error for completed runs on log–log axes. The labelled vertical lines show Euler and RK4 limits from the initial frozen Jacobian; they are local estimates, so the nonlinear sweep is needed to assess their relevance. Energy changes and failed-run statuses are in `stability_sweep.csv`.

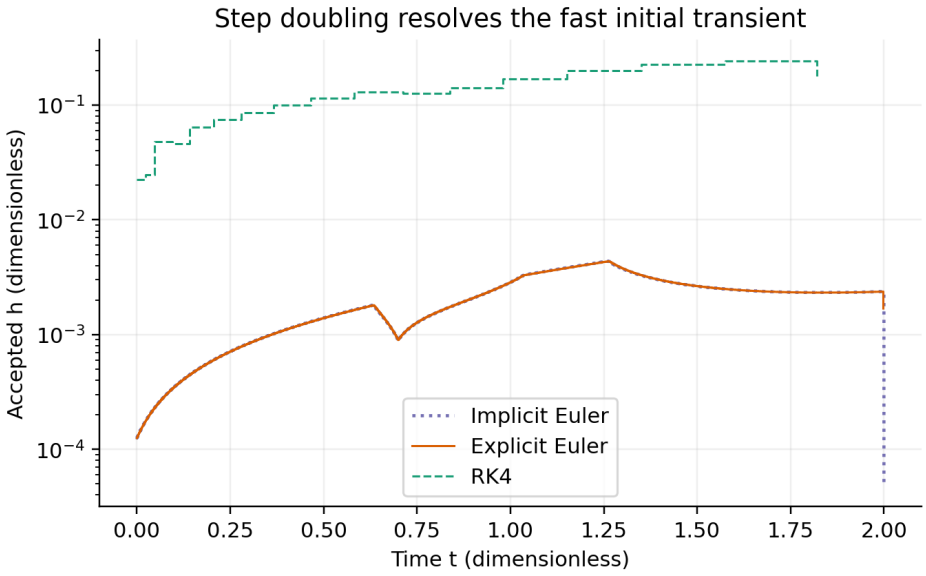


Figure 5: Accepted step length versus dimensionless time. Step doubling reduces the step during the rapid initial transient; the accepted-step data and estimated local error are in `adaptive_steps.csv`.

of $\mathrm{d}\Phi/\mathrm{d}t = -\|X(I - X^T X)\|_F^2$. Its maximum absolute residual is about 0.240, concentrated in the initial fast transient; the maximum residual scaled by $\max(1, |\mathrm{d}\Phi/\mathrm{d}t|)$ is 4.69×10^{-4}. This is a discrete diagnostic, not a direct test that the continuous identity fails.

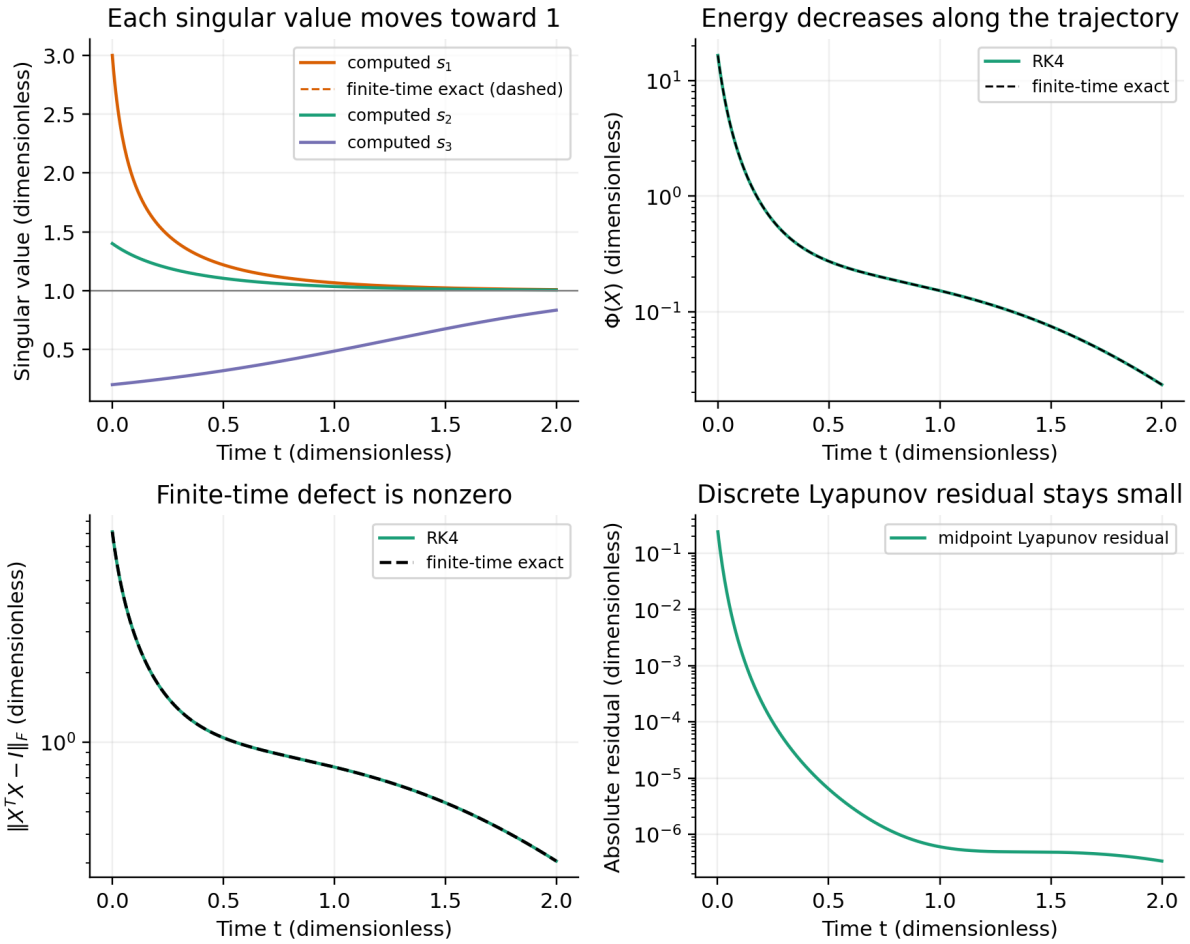


Figure 6: Singular values, Lyapunov energy, finite-time orthogonality defect, and the discrete identity residual along a resolved RK4 trajectory. Dashed curves give the finite-time exact SVD reference for the first three panels; the fourth panel explicitly labels the midpoint Lyapunov residual. Their overlap confirms accuracy while the nonzero exact defect explains why finite-time orthogonality is not expected.

Cost at comparable accuracy. We select fixed grids that yield terminal full-matrix errors within 5.4% of one another, then count RHS evaluations as the common work measure. Table 4 and Fig. 7 show that RK4 needs fewer RHS calls for this target, while Implicit Euler incurs many nonlinear-solver calls. Jacobian evaluations and Newton updates are reported separately; the counts do not include matrix factorization, Python overhead, or plotting and should not be interpreted as wall-clock timings.

Rank-deficient initial state. Replacing the first initial singular value by zero and integrating over $[0, 8]$ with RK4 at $h = 0.005$ leaves the smallest computed singular value below 7.1×10^{-15} throughout the stored path. The two nonzero terminal singular values are approximately 1.0000000500 and 1.0000000276; the final error against the finite-time exact matrix is 3.86×10^{-14}. The final orthogonality defect is approximately one, consistent with a rank-two partial isometry rather than an orthogonal limit (Fig. 8).

Method	N	Terminal error	RHS calls	Jacobian/Newton updates
Explicit Euler	160	9.2066×10^{-4}	160	0/0
RK4	17	8.8502×10^{-4}	68	0/0
Implicit Euler	160	9.3194×10^{-4}	862	351/351

Table 4: Counted work at closely matched terminal error.

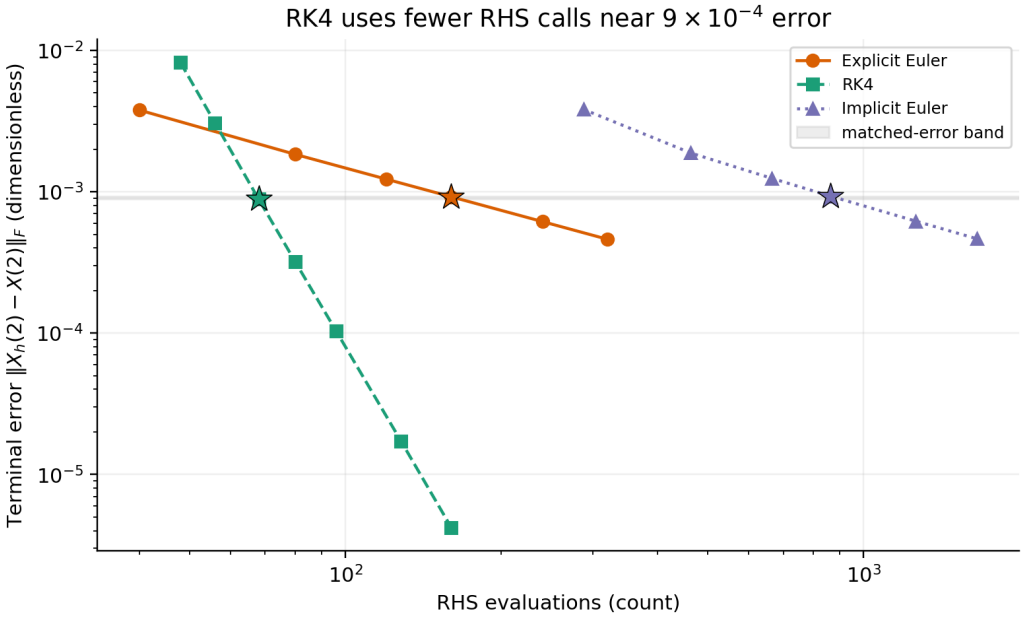


Figure 7: Achieved terminal full-matrix error versus counted RHS calls over several fixed grids. Stars mark the three table entries with errors near 9×10^{-4}; the grey band spans their actual errors. RK4 uses 68 RHS calls at this target, versus 160 and 862 for Explicit and Implicit Euler. Jacobian and Newton costs are reported separately in `cost_accuracy.csv`, so RHS count is not wall-clock time.

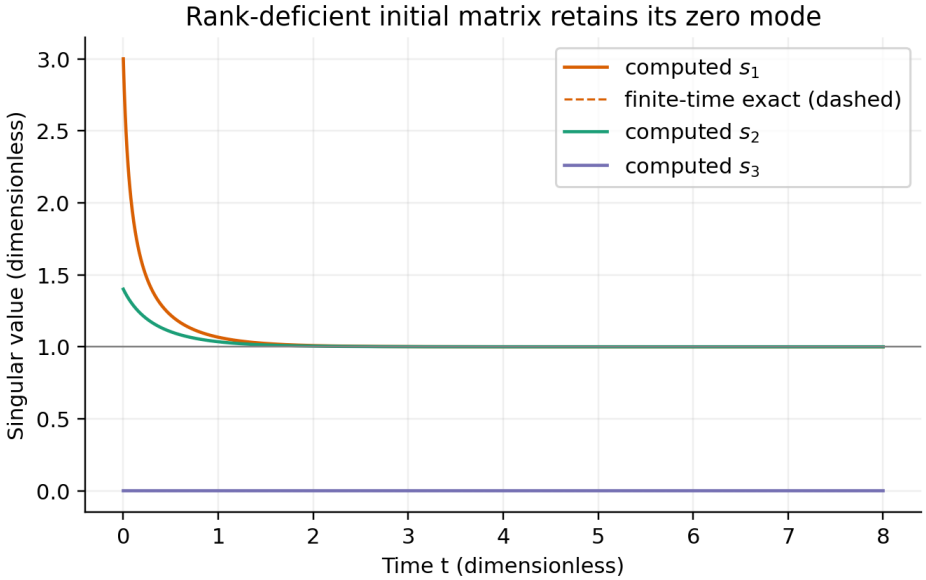


Figure 8: The zero singular mode remains at round-off scale while the other modes approach one, agreeing with the dashed exact SVD curves. This distinguishes the partial-isometry limit from an orthogonal one. Values are in `rank_deficient.csv`.

4.5 Verification Coverage

The runnable checks in `test/` comprise thirteen core test functions. They check the model and full implicit-residual Jacobians by centred finite differences, both complete eigenvalue spectra, the exact matrix against tight-tolerance Radau, and a diagonal scalar sanity case. All three methods are checked for fixed-grid order, genuinely varying adaptive steps and accepted normalized estimated local errors at most one. Their well-resolved trajectories are also checked for singular values approaching one without crossing and non-increasing Lyapunov energy. Additional edge checks compare loose and tight adaptive tolerances against the exact global error, expose singular-value crossing for a large Explicit Euler step, and exercise Newton failure followed by adaptive step halving and retry. The rank-deficient check verifies the zero mode and convergence toward the rank-two partial isometry. The checks pass on the recorded environment.

The RK4 order test uses $N = 640$ and 1280, avoiding the finest experiment grid whose error is nearer round-off scale. Passing these checks supports the tested configurations; it does not turn the local adaptive estimator into a universal global error bound. The full numerical evidence and CSV interpretation are supplied in Appendix C.

4.6 Limitations and Robustness

The initial $h\lambda$ overlay is a frozen-Jacobian warning, not a sufficient global nonlinear stability condition. An accepted step-doubling estimate is likewise a local indicator rather than a proof of global accuracy or stability. Large steps can destroy monotonicity of Φ or cause an explicit run to fail; Newton may also fail to find a decreasing candidate. The fixed-grid solver reports such failures instead of changing the grid used to estimate order, whereas the adaptive solver retries from the last accepted state. Our matched-accuracy cost study counts RHS calls and reports Newton work separately; a timing study would be needed to compare actual runtime on a specified machine.

5 Advanced Method: Two-Stage Gauss IRK4

5.1 Method Formulation

As an additional advanced comparison, we consider the classical two-stage, fourth-order Gauss–Legendre implicit Runge–Kutta method described by Butcher [4], referred to here as Gauss IRK4. The coefficients used below are the standard two-stage Gauss coefficients given in Chapter 3, Section 34.2 of Butcher's text.

This is a classical Gauss collocation method rather than the four-stage classical explicit RK4 scheme. The method is included as an extension to the three baseline methods studied in Sections 2–4.

Using the same column-wise vectorisation as in the main analysis, let

$$y = \mathrm{vec}(X) \in \mathbb{R}^9, \qquad f(y) = \mathrm{vec}\big[X(I - X^T X)\big] .$$

The two Gauss nodes are

$$c_1 = \frac{1}{2} - \frac{\sqrt{3}}{6}, \qquad c_2 = \frac{1}{2} + \frac{\sqrt{3}}{6},$$

with weights

$$b_1 = b_2 = \frac{1}{2},$$

and coefficient matrix

$$A = \begin{pmatrix} \frac{1}{4} & \frac{1}{4} - \frac{\sqrt{3}}{6} \\ \frac{1}{4} + \frac{\sqrt{3}}{6} & \frac{1}{4} \end{pmatrix}.$$

For one time step of size h, the stage values satisfy

$$Y_i = y_n + h \sum_{j=1}^{2} a_{ij} f(Y_j), \qquad i = 1, 2, \tag{117}$$

and the new solution is

$$y_{n+1} = y_n + \frac{h}{2} \left[f(Y_1) + f(Y_2) \right]. \tag{118}$$

Because the two stages depend on each other, they must be solved as one coupled nonlinear system rather than as two independent nine-dimensional problems.

5.2 Solution of the Implicit Stages

Define the coupled residuals

$$F_i(Y_1, Y_2) = Y_i - y_n - h \sum_{j=1}^{2} a_{ij} f(Y_j), \qquad i = 1, 2.$$

The (i, j) block of the Newton Jacobian is

$$\frac{\partial F_i}{\partial Y_j} = \delta_{ij} I_9 - h\, a_{ij} J_f(Y_j).$$

Therefore, each Newton iteration requires the solution of an 18×18 linear system. The implementation uses the predictors

$$Y_i^{(0)} = y_n + c_i h f(y_n),$$

followed by damped Newton iteration. A trial correction is accepted only when the full coupled stage residual remains finite and decreases.

The stopping condition is

$$\|F\|_\infty \le 10^{-12} \left(1 + \|Y\|_\infty\right).$$

This tolerance controls the residual of the implicit stage equations. It should not be interpreted as a bound on the global error of the numerical solution.

5.3 Order and Convergence

The classical two-stage Gauss collocation method has order

$$p = 2s = 4.$$

For exactly solved stage equations, the one-step defect is $O(h^5)$, and the corresponding per-unit-step local truncation error is $O(h^4)$. Since the vector field

$$f(X) = X(I - X^T X)$$

is polynomial, it is smooth on bounded sets relevant to the finite-time integration considered here.

To connect the method with the global error framework used in Section 2, suppose that f is Lipschitz with constant L on a bounded neighbourhood containing the relevant stage states. For sufficiently small h satisfying

$$h \|A\|_\infty L < 1,$$

the coupled stage equations have a locally unique solution in that neighbourhood, and the resulting one-step map has a uniform local Lipschitz bound. The discrete Grönwall argument used

previously then gives global fourth-order convergence over a fixed finite time interval, provided that the exact and numerical trajectories remain in the same bounded neighbourhood.

In practice, the stage equations are solved only approximately. If the stage residual is $\varepsilon(h)$ and the local inverse bound remains valid, the induced stage error is $O(\varepsilon)$ and the accumulated global contribution is also $O(\varepsilon)$. Therefore, a strict asymptotic fourth-order statement as $h \to 0$ requires the stage residual to decrease appropriately with h. The implementation uses the fixed threshold 10^{-12}, so the numerical evidence below should be interpreted as a finite-grid verification rather than as an independent asymptotic theorem.

5.4 Absolute Stability

For the scalar test equation

$$y' = \lambda y, \qquad z = h\lambda,$$

the amplification factor of the two-stage Gauss method is

$$R_G(z) = \frac{1 + \frac{z}{2} + \frac{z^2}{12}}{1 - \frac{z}{2} + \frac{z^2}{12}}. \tag{119}$$

Let

$$N(z) = 1 + \frac{z}{2} + \frac{z^2}{12}, \qquad D(z) = 1 - \frac{z}{2} + \frac{z^2}{12}.$$

Then

$$|D(z)|^2 - |N(z)|^2 = -2\,\mathrm{Re}(z)\left(1 + \frac{|z|^2}{12}\right).$$

Hence, when $\mathrm{Re}(z) \le 0$,

$$|R_G(z)| \le 1,$$

so the method is A-stable. This is the classical A-stability property of Gauss methods; see Butcher, Chapter 3, Section 35.3 [4].

However,

$$R_G(z) \to 1 \qquad \text{as } z \to -\infty$$

along the negative real axis. Therefore, the Gauss method is not L-stable. In particular, very strongly damped modes are not necessarily suppressed as aggressively as they are under Implicit Euler.

For the initial frozen Jacobian of the present problem, the fastest negative mode is approximately $\lambda = -26$. With $h = 0.08$,

$$z = -2.08,$$

for which the Gauss amplification factor is approximately 0.1335. This frozen-spectrum calculation is only a local diagnostic and should not be interpreted as a complete nonlinear stability proof. Figure 9 shows the negative-real-axis amplification factors; A-stability on the full left half-plane follows from the preceding identity rather than from this one-dimensional plot.

5.5 Numerical Comparison

The Gauss IRK4 implementation was tested on the same initial matrix and time interval $[0, 2]$ used in the main experiments. The terminal Frobenius-norm error was again measured against the exact finite-time SVD solution.

Using the refined grids

$$N = 64,\ 128,\ 256,$$

the fitted convergence slope for Gauss IRK4 is approximately

$$3.9674,$$

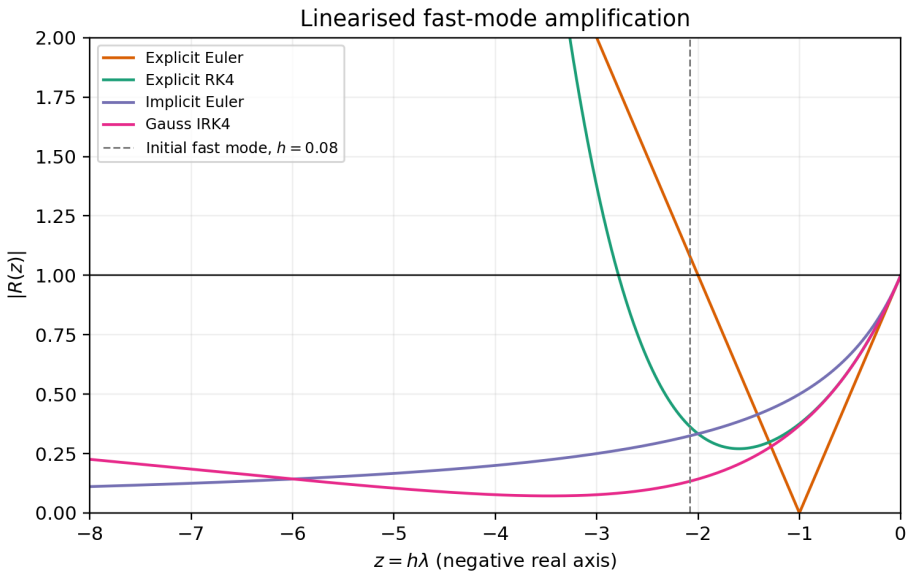


Figure 9: Comparison of the scalar stability amplification factors along the negative real axis. The dashed reference marks the initial frozen mode $z = -26 \times 0.08 = -2.08$. The Gauss IRK4 curve illustrates its negative-axis damping and its lack of L-stability.

which is consistent with the expected fourth-order behaviour on the tested finite grids.

For comparison, the corresponding fitted slopes in the same comparison script are approximately

$$1.0101, \quad 3.9274, \quad 1.0137$$

for Explicit Euler, classical RK4, and Implicit Euler, respectively. Figure 10 compares these fitted orders and terminal errors on the same axes.

A matched-accuracy comparison was also carried out. Classical RK4 with $N = 75$ gives terminal error

$$5.09443 \times 10^{-8}$$

using 300 right-hand-side evaluations. Gauss IRK4 with $N = 64$ gives terminal error

$$5.07898 \times 10^{-8},$$

using 720 right-hand-side evaluations, 264 Jacobian evaluations, and 132 Newton iterations. The two terminal errors differ by approximately 0.3%.

Thus, for this 3×3 benchmark and this accuracy level, Gauss IRK4 does not reduce the number of right-hand-side evaluations relative to classical RK4. In addition, the evaluation counts do not include the cost of Jacobian assembly, the 18×18 linear solves, or Python overhead, so they should not be interpreted as direct wall-clock timing results. Figure 11 displays achieved error against the counted right-hand-side work for all four methods.

On the common grid $h = 0.1$, the sampled Lyapunov energy decreases at each stored time for all four methods. This is an observation from the present experiment rather than a general proof that Gauss IRK4 preserves energy monotonicity for arbitrary step sizes (Fig. 12).

Finally, tightening the stage residual threshold from 10^{-12} to 10^{-14} changes the terminal solution by approximately

$$4.44 \times 10^{-16}, \qquad 1.50 \times 10^{-13}, \qquad 3.47 \times 10^{-14}$$

for $N = 64, 128, 256$, respectively. These changes are much smaller than the corresponding time-discretisation errors, supporting the use of the fixed Newton tolerance over the tested grid range.

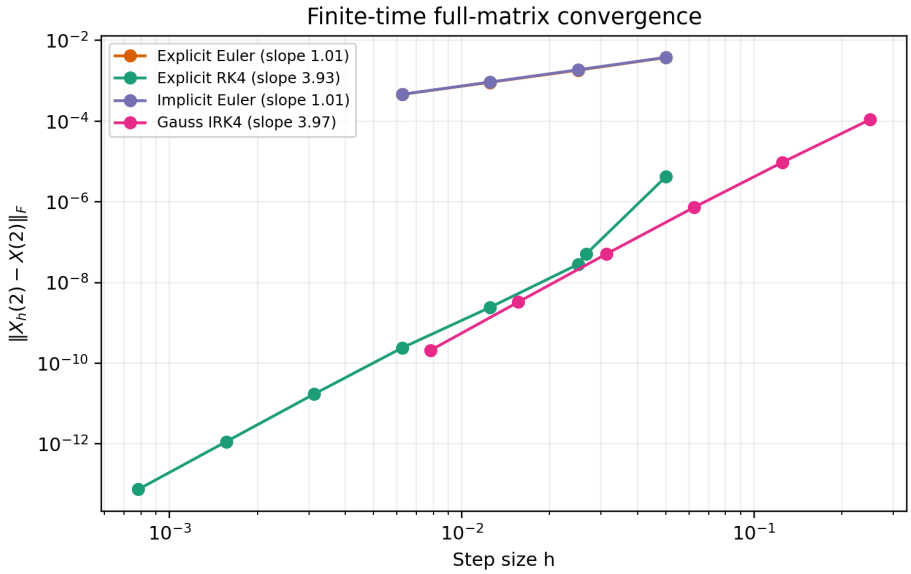


Figure 10: Terminal Frobenius-norm error versus step size for the four numerical methods. The fitted Gauss IRK4 slope on the refined grids is approximately 3.9674, consistent with fourth-order behaviour on the tested finite grids.

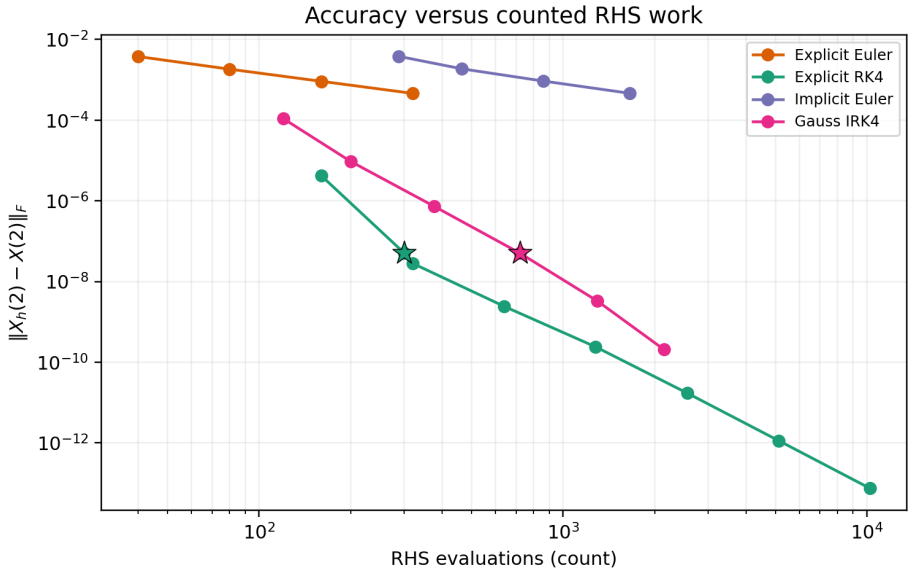


Figure 11: Terminal error versus counted right-hand-side evaluations. The highlighted RK4 and Gauss IRK4 runs have terminal errors differing by approximately 0.3%. The plotted work measure does not include Jacobian assembly, linear solves, or Python overhead.

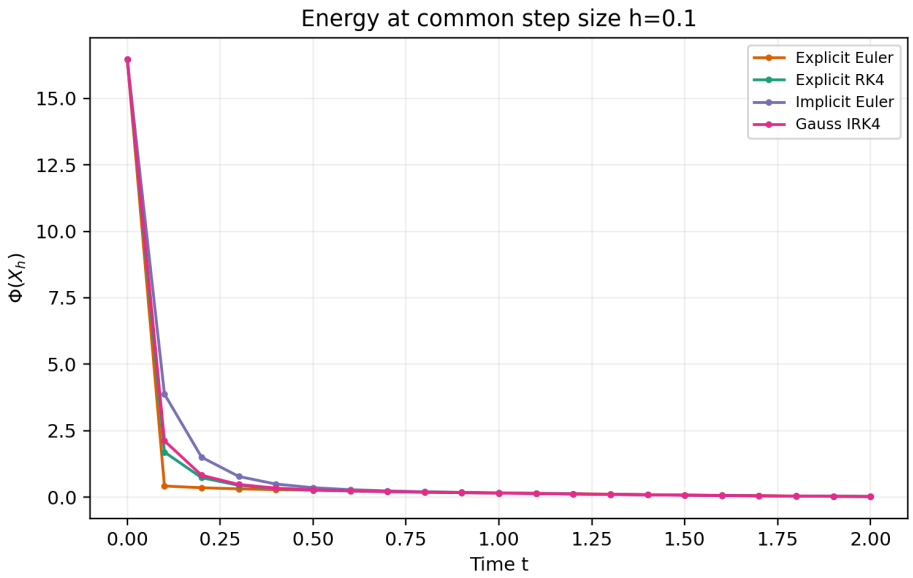


Figure 12: Sampled Lyapunov energy trajectories for the four methods on the common grid $h = 0.1$. All sampled trajectories decrease in this experiment; this is an empirical observation rather than a general monotonicity theorem.

The numerical values reported in this subsection are generated by the advanced-method comparison scripts. At the project level, the convergence slopes, matched-accuracy errors, and work counts are recorded in `data/advanced_irk4/convergence_cost.csv` and `data/advanced_irk4/summary.json`. The energy trajectories shown in Figure 12 are recorded in `data/advanced_irk4/energy_trajectories.csv`.

6 Conclusions

This project investigated the numerical solution of the polar-factor matrix gradient flow

$$\dot{\mathbf{X}} = \mathbf{X}\left(\mathbf{I}_n - \mathbf{X}^T\mathbf{X}\right)$$

using Explicit Euler, classical fourth-order Runge–Kutta, and Implicit Euler with damped Newton iteration as baseline methods, followed by a two-stage Gauss implicit Runge–Kutta extension. The study combined convergence analysis, local stability diagnostics, adaptive step-size control, geometric checks, and matched-accuracy work comparisons.

The fixed-grid experiments reproduce the expected convergence orders. Explicit Euler and Implicit Euler give fitted slopes of approximately 1.010 and 1.014, respectively, while RK4 gives a fitted slope of approximately 3.927 on the refined grids. These measurements are consistent with first-order global convergence for the Euler methods and fourth-order convergence for RK4. The exact finite-time SVD solution is independently supported by a tight-tolerance Radau computation, whose terminal Frobenius-norm difference from the analytic matrix is 1.16×10^{-13}.

The stability experiments also support the theoretical analysis while illustrating its limitations. The initial frozen-Jacobian estimates are approximately

$$h_{\text{crit}}^{\text{EE}} \approx 0.0769, \qquad h_{\text{crit}}^{\text{RK4}} \approx 0.1071.$$

Direct perturbation and step-size experiments show increased amplification and eventual failure when sufficiently large explicit steps are used. However, the experiments also demonstrate that these values should be interpreted as local frozen-Jacobian warnings rather than exact nonlinear

global stability thresholds. Implicit Euler is more robust on the tested coarse grids, although this additional stability does not by itself guarantee comparable accuracy.

The step-doubling adaptive controller operates as intended for all three base methods. Accepted step sizes vary during the integration and become smaller during the rapid initial transient. With the common tolerance settings used in the experiments, the maximum accepted normalized local error remains below one for each solver. These local estimates are not treated as bounds on the terminal global error, which is measured separately against the exact SVD solution.

The geometric experiments are consistent with the structure of the continuous matrix flow. On a resolved RK4 trajectory, all sampled singular values move toward one without crossing, and the Lyapunov functional decreases at every stored step. At $t = 2$, the numerical orthogonality defect is 0.305932824226, compared with the exact finite-time value 0.305932824210. This agreement confirms that the nonzero defect at finite time is a property of the exact trajectory rather than a numerical failure.

The matched-accuracy comparison provides a further distinction between the three methods. For terminal errors close to 9×10^{-4}, RK4 requires 68 right-hand-side evaluations, compared with 160 for Explicit Euler and 862 for Implicit Euler. The implicit run additionally requires 351 Jacobian evaluations and Newton updates. Under the chosen algorithmic work measure, RK4 therefore reaches the selected accuracy with the fewest right-hand-side evaluations. These counts exclude matrix factorisation, Python overhead, and plotting, so they should not be interpreted as direct wall-clock timing measurements.

The advanced comparison in Section 5 adds a fourth-order Gauss implicit Runge–Kutta method. Its measured refined-grid slope is approximately 3.9674. The method is A-stable but not L-stable; at a matched terminal error near 5.1×10^{-8}, it uses 720 right-hand-side evaluations compared with 300 for classical RK4. Thus this extension does not improve the counted RHS work for the tested benchmark and accuracy, although its stability properties differ from those of explicit RK4.

The rank-deficient experiment confirms another structural property of the model. When the first initial singular value is replaced by zero, the smallest numerical singular value remains at round-off scale while the other two approach one. The resulting state therefore approaches a rank-two partial isometry rather than an orthogonal matrix, in agreement with the exact finite-time solution.

Overall, the numerical evidence is consistent with the theoretical analysis developed in Sections 2 and 5. RK4 provides the most favourable accuracy–work balance among the tested configurations, while Implicit Euler provides greater robustness for large steps at the additional cost of nonlinear solves. Explicit Euler remains a useful first-order baseline but is more strongly restricted by accuracy and stability.

Several limitations should be retained when interpreting these results. The frozen-Jacobian stability analysis is local rather than a complete nonlinear stability proof, and the adaptive error estimator controls a local quantity rather than the global solution error. In addition, the cost study uses algorithmic evaluation counts rather than measured runtime on a fixed machine. Nevertheless, agreement among the analytic SVD solution, the independent Radau reference, the geometric diagnostics, and the automated validation tests provides a consistent basis for confidence in the reported numerical results.

A natural direction for future work is to extend the comparison to other higher-order implicit methods, especially L-stable stiff solvers, larger matrix dimensions, and measured wall-clock performance. A more complete nonlinear stability analysis beyond the frozen-Jacobian approximation would also be useful.

References

[1] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed., Springer, 1993. DOI: 10.1007/978-3-540-78862-1.

[2] N. J. Higham, "Computing the Polar Decomposition—with Applications," *SIAM Journal on Scientific and Statistical Computing*, 1986. DOI: 10.1137/0907079.

[3] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020. DOI: 10.1038/s41592-019-0686-2.

[4] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed., Wiley, 2016.

Appendices

A AI Transparency Log

Tools Used

- **ChatGPT (OpenAI):** Used to clarify project tasks, review report structure, improve wording, and assist with final consistency checks.

AI-Assisted Work and Claim Checking

AI-1: Report structure and completeness. AI was used to help identify missing or incomplete final-delivery items, including appendices, ICS, version documentation, and report consistency.

Action taken: These suggestions were checked against the project brief and report template before being applied.

AI-2: Numerical results and Section 2 wording. AI was used to help summarise existing numerical results and clarify several wording issues in Section 2, including local error terminology and the role of the Newton tolerance in Implicit Euler.

Claim checking: Numerical values, including the reported convergence slopes and the SVD/Radau terminal difference, were checked against the team's existing outputs. The Section 2 clarification was also checked against the existing derivation and numerical experiments before being added.

Action taken: AI-generated wording was revised so that it matched the mathematical notation, numerical evidence, and actual scope of the project.

Changes Made to AI Output

AI-generated text was reviewed and edited before inclusion in the report. No new numerical results were introduced by AI, and numerical claims were kept only when they could be checked against the team's existing code outputs, figures, data, or derivations.

Intellectual Ownership Declaration

The mathematical model, numerical methods, implementation, experiments, validation data, figures, and project results are the team's own work. Generative AI was used only as a supporting tool for clarification, checking, organisation, and drafting assistance. Responsibility for the final content remains with the team.

B Individual Contribution Statements

The statements below describe work at the time each member prepared their individual record. Historical page and test counts refer to those dated builds; the current reproduction checklist is in `report_v3/checklist/README.md`.

Tianrui Ji (Student ID: 2361367)

Role this project: Lead developer of the Python implementation and numerical experiments for Topic 5.

The specific things I did this project:

1. I implemented, revised, and reviewed the matrix ODE model in `Project1/code/model.py` and its Chinese counterpart. This includes matrix vectorisation, the right-hand side, the Jacobian, the finite-time exact solution, and diagnostic quantities used to check the numerical solutions.
2. I implemented, revised, and reviewed the three numerical methods in `Project1/code/solvers.py` and its Chinese counterpart: explicit Euler, classical RK4, and implicit Euler with a Newton solver. I also worked on the fixed-step and adaptive drivers, including the implicit-step residual and the handling of failed nonlinear solves.
3. I implemented and revised the reproducible experiments in `Project1/code/experiments.py` and `run_all.py`, including convergence, stability, trajectory, cost–accuracy, adaptive-step, and rank-deficient diagnostics. I reviewed the generated data and figures against the finite-time exact solution and documented the code and figures in the English and Chinese README files.

What I would do differently: I would agree on the definitions of the implicit-step residual and error measures with the theory and testing members before implementation. I would also record parameter choices and the mapping between data and figures when each experiment is created.

Anything I should have asked for help with earlier: I should have requested an independent review of the Newton solver and of how the stability analysis supports the numerical conclusions earlier in the project.

AI use: AI assisted with implementation and review, as disclosed in the team's AI Transparency Log. I reviewed and revised the submitted code and take responsibility for explaining its methods, verification evidence, and limitations.

Jingyan Wei (Student ID: 2361411)

Role: Mathematical Theory (Schemes & Stability Lead)

1. My specific contributions to this project:

I authored the discrete formulation and linear/nonlinear stability foundations in Section 2 (Sections 2.1 and 2.3) now in `Project1/report_v3/sections/section2.tex`; an earlier draft is preserved as `Project1/report_v1/Section2Draft_v5.tex`. Specifically, I derived the step-update relations for Explicit Euler (Eq. 6–7 in Section2Draft_v5), classical RK4 with its strictly lower-triangular Butcher tableau (Eq. 12–19), and Implicit Euler (Eq. 20–21). For the implicit solver, I derived the non-commutative matrix Fréchet derivative $\mathcal{D}f(X)[H] = H(I_n - X^TX) - X(H^TX + X^TH)$ (Eq. 25) and the residual Jacobian $J_F(w) = I_{n^2} - hJ_f(w)$ (Eq. 24, 28), formalising the damped Newton iteration with residual-decreasing backtracking line search (Algorithm 1). Furthermore, I proved the analytical stability boundaries for the scalar Dahlquist equation: showing that $R_{\mathrm{RK4}}(-a) = -1$ has no real roots (Eq. 76) and solving the cubic equation $q(a) = a^3 - 4a^2 + 12a - 24 = 0$ to establish the negative real intercept $a^* \approx 2.78529$ (Eq. 77) and imaginary limit $[-2\sqrt{2}, 2\sqrt{2}]$ (Eq. 78). For Topic 5, I transformed the linearised operator into SVD coordinates (Eq. 91), rigorously separating contracting normal modes $\lambda_{ij}^{\mathrm{sym}}$ from neutral tangent modes $\lambda_{ij}^{\mathrm{skew}}$ (Eq. 92–95), determined the full initial spectrum $\{-26.0, \ldots, +0.88\}$ (Eq. 96), calculated the theoretical explicit step limits $h_{\mathrm{crit}}^{\mathrm{EE}} \approx 0.0769$ and $h_{\mathrm{crit}}^{\mathrm{RK4}} \approx 0.1071$ (Eq. 97–98), and authored the Mandatory Analytical Disclaimer governing local frozen spectra.

2. The hardest conceptual obstacle I encountered:

The hardest mathematical challenge was deriving the problem-specific Fréchet derivative and Jacobian spectrum for the matrix gradient flow. In standard scalar calculus, differentiating $x(1 - x^2)$ yields $1 - 3x^2$, but for the matrix-valued flow $\dot{X} = X(I_n - X^TX)$, matrix multiplication is strictly non-commutative. When taking directional variations, the perturbation in the quadratic term expands as $\mathcal{D}(X^TX)[H] = H^TX + X^TH$ rather than $2X^TH$. Projecting this onto the tangent space of the SVD manifold required diagonalising a coupled 2×2 block system for every coordinate pair $(i < j)$, where I had to analytically demonstrate that symmetric perturbations induce contraction ($\lambda = -2$) while skew-symmetric variations align with the Lie algebra $\mathfrak{so}(n)$ of neutral tangent rotations ($\lambda = 0$).

3. One piece of AI-generated output my team used, and what I changed:

Generative AI was used to detail the RK4 negative-real-axis stability proof and analyse numerical stability. When querying AI to evaluate explicit stability on Topic 5, the raw output claimed that ensuring the scaled frozen eigenvalues satisfied $|1 + h\lambda_j| \leq 1$ ($h < 2/26 \approx 0.0769$) provided a sufficient mathematical proof of global numerical stability over $t \in [0, 2]$. I audited this claim and identified that because the Jacobian $J_f(X(t))$ drifts continuously along the nonlinear trajectory, frozen eigenvalues only provide a necessary local linearised indicator rather than a global nonlinear certificate. I revised Section 2.3.4 to introduce the Mandatory Analytical Disclaimer and specified the step-sweep perturbation protocol to empirically test true nonlinear blow-up boundaries.

4. One thing I still do not fully understand:

I do not yet have a sharp nonlinear perturbation bound along the entire trajectory. The vectorised Jacobian $J_f(X)$ is symmetric at each state, but its eigenvalues and eigendirections change as $X(t)$ evolves. Pointwise frozen spectra alone therefore do not provide the global time-dependent bound that would quantify perturbation growth over $[0, 2]$.

Signature: _____ **Date:** _____

Meichen Ding (Student ID: 2362344)

Role: Mathematical Theory Lead

1. My specific contributions to this project:

I authored the analytical consistency, adaptivity, and convergence foundations in Section 2 (Sections 2.2 and 2.4) now in `Project1/report_v3/sections/section2.tex`; an earlier draft is preserved as `Project1/report_v1/Section2Draft_v5.tex`. Specifically, I derived the local truncation errors and consistency orders for Explicit/Implicit Euler and RK4 (Eq. 37, 43, 45–52 in Section2Draft_v5) under Picard–Lindelöf Lipschitz continuity and small-step contraction $hL < 1$ (Eq. 42). From first principles, I derived the step-doubling Richardson extrapolation error estimator, proving that the exact continuous state cancels out to yield the unnormalised defect denominator $2^p - 1$ (Eq. 57–58) and the scale-invariant step-size update controller with optimal exponent $-1/(p+1)$ (Eq. 62). Furthermore, I formulated the discrete error recurrence (Eq. 104–105) and proved the continuous-type discrete Grönwall global upper bound $\|e_n\| \leq \frac{e^{\Lambda(T-t_0)}-1}{\Lambda}\tau_{\max}(h)$ (Eq. 109), establishing why local defects of order $\mathcal{O}(h^{p+1})$ accumulate over $N = \mathcal{O}(h^{-1})$ steps into $\mathcal{O}(h^p)$ global convergence (Eq. 110–111), while proving the zero-stability degeneracy ($\rho(\zeta) = \zeta - 1 = 0$) of single-step methods and providing the theoretical derivation for empirical log-log slope regression (Eq. 114–115).

2. The hardest conceptual obstacle I encountered:

The hardest mathematical challenge was reconciling why accepting the fine sub-step solution y_f in our step-doubling adaptive controller did not elevate the asymptotic global convergence order from p to $p+1$ in empirical log-log regressions, despite the controller utilizing a local extrapolation error formula. I overcame this by writing out the full Taylor remainder expansions for both the coarse step y_c and consecutive half-steps y_f, which mathematically proved that combining two half-steps merely attenuates the leading principal error coefficient by a factor of 2^{-p} without satisfying the underlying Butcher algebraic tree conditions required for order $p+1$, thereby clarifying that local error control must strictly follow the exponent $-1/(p+1)$.

3. One piece of AI-generated output my team used, and what I changed:

Generative AI was consulted to revise the Implicit Euler local-error argument and adaptive stepping. When querying AI for the adaptive step-size update rule, the raw output proposed a textbook formula $h_{\text{new}} = h \cdot E^{-1/p}$. I audited this claim and identified that it conflated accumulated global discretization error $\mathcal{O}(h^p)$ with single-step local truncation defect $\mathcal{O}(h^{p+1})$. I corrected the resizing exponent to $-1/(p+1)$ in Section 2.2.4 (Eq. 62), derived the rigorous step-doubling cancellation denominator $2^p - 1$ (1 for Euler, 15 for RK4), and introduced the safety margin $S = 0.9$ alongside limiter bounds $[r_{\min}, r_{\max}]$ to prevent step-chattering during the initial transient.

4. One thing I still do not fully understand:

I do not yet fully understand how to construct a rigorous non-autonomous matrix logarithmic norm (one-sided Lipschitz dissipative constant) for the nonlinear polar-factor gradient flow $\dot{X} = X(I_n - X^T X)$ over the entire finite-time trajectory $t \in [0, 2]$, which would provide a unified global nonlinear contraction bound to theoretically explain why the discrete Lyapunov residual exhibits localized pre-asymptotic spikes during the fast transient even when the continuous Lyapunov functional $\Phi(X)$ decreases monotonically.

Signature: _____ **Date:** _____

Jiasheng Xu (Student ID: 2251640)

Role this project: Role 5 — Testing and Validation.

1. My specific contributions to this project:

1. I added and migrated three edge-case validation tests in `Project1/test/test_validation.py`: tighter adaptive tolerances must reduce global error, an oversized explicit step exposes singular-value crossing, and Newton failure triggers adaptive retry. These changes are traceable to commit `c4ca4ee` and merge commit `b28fe39`.

2. I independently ran both the English and Chinese entry points and the complete test suite, obtaining 14/14 passing tests. I recorded the clean reproduction in `Project1/report_v3/checklist/VALIDATION_RUN_2026-10-03.md` (commit `06461a9`). After review, I corrected the Radau comparison to 1.17×10^{-13} at three significant figures and regenerated and independently checked all 41 manifest hashes in the Windows submission checkout using the documented procedure (commit `838d938`).

3. After the validation merge, I rebuilt the report PDF from `Project1/report_v3/report.tex`, checked all 32 pages, and confirmed that Section 4 reports fourteen tests and that the contents, references, and appendices are complete. I also updated the change log, validation checklist, and affected manifest hashes. This report-integration work was merged through PR #3 (merge commit `e6913e6`).

2. The hardest conceptual obstacle I encountered: The main challenge was separating a local step-doubling error estimate from accumulated global error. I addressed it by comparing against the exact solution and turning the expected global-error reduction into a deterministic regression test rather than treating one accepted local step as evidence of global accuracy.

What I would do differently: I would begin validation from the agreed current `main` branch, pin dependency versions, and define rounding and manifest-hash rules before generating final evidence. This would avoid the path migration and last-minute rehashing required here.

Anything I should have asked for help with earlier: I should have asked the report integrator to confirm the authoritative report directory and the freeze point for final artefacts before preparing the original test PR.

3. One piece of AI-generated output my team used, and what I changed: As recorded in Appendix A, AI helped check deliverable completeness and organise a draft of the validation evidence. I revised that draft after manual reproduction: I corrected the Radau rounding, distinguished close agreement from exact matching, ran the complete suite in a clean environment, and independently verified the documented SHA-256 results before submission.

4. One thing I still do not fully understand: I do not yet have a rigorous global-error bound for this nonlinear matrix flow that starts from the local step-doubling estimator while also accounting for rejected Newton steps.

Signature: Jiasheng Xu **Date:** 4 October 2026

Han Liu (Student ID: 2364393)

1. My specific contributions to this project:

I was mainly responsible for the final-stage writing and integration of the report. I completed and refined Section 1 (Background) and Section 5 (Conclusions), including the project motivation, overall context, and the synthesis of the main numerical findings. I also checked the completeness and consistency of the final report files, updated the appendices and version documentation, and helped integrate the existing Sections 2–4, figures, data, and references into the complete report. In the final checking stage, I also reviewed several wording and notation issues in Section 2 and clarified the discussion of the Implicit Euler Newton tolerance and local error terminology.

2. The hardest conceptual obstacle I encountered:

The most difficult part for me was understanding how the theoretical error statements in Section 2 should be connected to the actual numerical implementation. In particular, I had to distinguish the per-unit-step local truncation error from the unnormalised local one-step defect, and understand why the convergence proof for Implicit Euler assumes a sufficiently accurate nonlinear solve. I addressed this by comparing the mathematical definitions, the Newton stopping criterion used in the code, and the numerical convergence results before revising the wording in the report.

3. One piece of AI-generated output my team used, and what I changed:

As recorded in Appendix A, AI was used to help review the report structure and identify wording and completeness issues in the final document. One example was the suggested clarification of the Implicit Euler convergence discussion in Section 2. I checked the AI-assisted explanation against the existing derivation and numerical results, and revised the wording so that the report explicitly states that a fixed Newton tolerance is adequate for the tested finite range of step sizes but is not itself an asymptotic assumption as $h \to 0$. The final wording and its inclusion in the report were decided after this checking process.

4. One thing I still do not fully understand:

I still do not fully understand the more general theory describing how the accuracy of an inexact nonlinear solve affects the convergence order of an implicit time-stepping method beyond this specific Implicit Euler example. I understand the practical requirement that the nonlinear-solver error should be smaller than the time-discretisation error, but I would like to understand a more general rigorous analysis of this interaction.

Signature: _____ **Date:** _____

C Supplementary Figures and Data

C.1 Reproduction and Evidence Files

This appendix documents the six CSV datasets and `summary.json` used by the eight figures in Sections 2–4. Complete machine-readable files are bundled in `report_v3/data/`; the corresponding submitted images are in `report_v3/figures/`. The tables below are compact readable extracts from those files. They preserve the full evidence without replacing the CSV trajectories by rounded typeset values.

From the `Project1/` directory, run:

```
python -m pip install -r requirements.txt
python -B code/run_all.py
python -B -m unittest discover -s test -v
```

The run rebuilds project-level `figures/` and `data/`. The regenerated files also match this version's submitted snapshots. PNGs are exported at 220 DPI.

All model quantities, time, and step lengths are dimensionless. Evaluation and iteration fields are integer counts. The model is deterministic, so no random seed is needed. The stored singular values are in descending SVD order, $s_1 \geq s_2 \geq s_3$; this ordering should not be confused with the original diagonal input order $(0.2, 1.4, 3)$.

C.2 Data-to-Figure Map

Evidence file	Rows	Associated figure(s)	Claim supported
`convergence.csv`	15	`convergence.png`	Full-matrix errors, observed orders and fitted slopes.
`cost_accuracy.csv`	19	`cost_accuracy.png`	Achieved error and counted work at matched accuracy.
`stability_sweep.csv`	39	`stability_sweep.png`	Fast-direction amplification, geometry failures and failed runs.
`trajectory.csv`	401	`trajectory_diagnostics.png`	Singular values, energy, finite-time defect and Lyapunov residual.
`adaptive_steps.csv`	3375	`adaptive_steps.png`	Accepted time grid, varying step size and estimated local budget.
`rank_deficient.csv`	1601	`rank_deficient.png`	Numerical and exact singular values for the rank-two initial matrix.
`summary.json`	—	`frozen_spectrum.png`, `stability_regions.png`; all experiment summaries	Settings, software versions, complete spectra, thresholds and aggregate diagnostics.

Table 5: Every numerical evidence file and the figure it supports. Row counts exclude CSV headers. Stable filenames allow each reported claim to be traced to the same version's full dataset.

`stability_regions.png` is evaluated directly from the three analytic stability functions, with the initial spectrum overlaid. It therefore has no separate sampled-trajectory CSV. Its amplification formulas are in Section 2.3, the sampling grid and overlay step are in `code/experiments.py`, and its spectrum is recorded in `summary.json`. `frozen_spectrum.png` uses the same recorded spectrum at $h = 0.05$ and 0.10.

C.3 CSV Field Definitions and Missing Values

Convergence and cost. Both `convergence.csv` and `cost_accuracy.csv` contain `method`, `n_steps`, `h`, `terminal_frobenius_error`, `rhs_evaluations`, `jacobian_evaluations`, and `newton_iterations`. The terminal error is $\|X_h(2) - X_{\text{exact}}(2)\|_F$. The convergence file

additionally stores `observed_order`, computed as $\log_2(E_h/E_{h/2})$; the first row of each method leaves this field blank because no coarser comparison is available. The cost file instead stores the Boolean `matched_point`, which identifies the starred comparison points. RHS counts include nonlinear residual and line-search evaluations; Jacobian assembly, factorization and plotting are not included in this cost measure.

Stability sweep. `stability_sweep.csv` contains `method`, `h`, `n_steps`, `one_step_fast_perturbation_amplification`, `one_step_energy_change`, `max_energy_increase`, `singular_value_crossing`, `terminal_exact_error`, and `status`. Amplification is measured using an initial perturbation of size 10^{-7} in the fastest singular-vector direction. `status=diverged_or_failed` means the full run did not finish. Blank or `nan` measurements in that row are unavailable quantities, not zero error or zero amplification. The plotted curves use completed runs; failed attempts remain in the CSV for inspection.

Resolved trajectory. `trajectory.csv` stores `t`, `s1`, `s2`, `s3`, `energy`, `exact_energy`, `orthogonality_defect`, `exact_orthogonality_defect`, and `lyapunov_identity_residual_next_interval`. The singular values, energy and defect refer to the row time. The residual refers to the interval beginning at that row and is plotted at its midpoint:

$$D_n = \frac{\Phi(X_{n+1}) - \Phi(X_n)}{t_{n+1} - t_n} + \left\| \bar{X}_n (I - \bar{X}_n^T \bar{X}_n) \right\|_F^2, \qquad \bar{X}_n = \tfrac{1}{2}(X_n + X_{n+1}).$$

The final row has no following interval, so its residual is blank. The figure shows $|D_n|$; the summary additionally reports the maximum scaled residual with denominator $\max(1, |\dot{\Phi}(\bar{X}_n)|)$. This scaled quantity is not a pure relative error.

Adaptive steps. `adaptive_steps.csv` stores `method`, `t_start`, `h`, and `normalized_local_error`. Each row represents an accepted step from `t_start` to `t_start+h`. The error is the normalized step-doubling estimate E in Algorithm 2, not the global terminal error. Accepted rows satisfy $E \le 1$. The 3375 rows comprise 1680 Euler, 17 RK4 and 1678 Implicit Euler steps. Rejected trial counts and independently measured terminal errors are in `summary.json`. A final very short step may simply hit $t = 2$.

Rank-deficient trajectory. `rank_deficient.csv` contains `t`, `s1`, `s2`, `s3`, and `exact_s1`, `exact_s2`, `exact_s3`. These are the numerical and finite-time exact sorted singular values for $X_0 = R(0.4)\,\mathrm{diag}(0, 1.4, 3)R(-0.7)^T$, on $[0, 8]$ with $h = 0.005$. The zero mode is the third column in the sorted numerical spectrum.

C.4 Selected Numerical Evidence

Method	N	h	$E_h(2)$	Adjacent order
Explicit Euler	160	0.0125	9.2066×10^{-4}	0.9973
Explicit Euler	320	0.00625	4.6141×10^{-4}	0.9966
RK4	640	0.003125	1.7123×10^{-11}	3.8039
RK4	1280	0.0015625	1.1312×10^{-12}	3.9201
RK4	2560	0.00078125	7.3968×10^{-14}	3.9348
Implicit Euler	160	0.0125	9.3194×10^{-4}	1.0114
Implicit Euler	320	0.00625	4.6429×10^{-4}	1.0052

Table 6: Selected rows from `convergence.csv`. Adjacent orders refer to the preceding coarser row in the complete dataset. The finest RK4 error is near round-off scale; no plateau is inferred.

Method	Matched error	RHS calls	Jacobian/Newton updates
Explicit Euler	9.2066×10^{-4}	160	0/0
RK4	8.8502×10^{-4}	68	0/0
Implicit Euler	9.3194×10^{-4}	862	351/351

Table 7: The `matched_point=True` rows from `cost_accuracy.csv`. The largest error is approximately 5.3% above the smallest; this is a RHS-work comparison, not a runtime ranking.

Diagnostic from `summary.json`	Recorded value
SVD/Radau terminal disagreement	1.161×10^{-13}
RK4 finite-time orthogonality defect	0.305932824226
Exact finite-time orthogonality defect	0.305932824210
Largest absolute midpoint Lyapunov residual	0.239792
Largest scaled midpoint Lyapunov residual	4.6936×10^{-4}
Largest zero singular mode in rank-deficient run	7.0899×10^{-15}
Rank-deficient terminal exact-matrix error	3.8613×10^{-14}
Rank-deficient terminal orthogonality defect	1.0000000000000064

Table 8: Checks supporting the finite-time, Lyapunov and rank-deficient interpretations. The finite-time defect and discrete residual are not expected to be zero.

C.5 JSON Contents and Version Traceability

`summary.json` records the benchmark, interval and Newton residual tolerance. It also records the software versions, convergence fit ranges and slope standard errors, work counts at matched accuracy, Radau settings, geometric checks, full initial and equilibrium spectra, local step-size estimates, failed sweep runs, adaptive statistics and rank-deficient diagnostics.

The recorded environment is Python 3.12.4, NumPy 1.26.4, SciPy 1.13.1 and Matplotlib 3.8.4. The initial full spectrum is

$$(-26, -14.16, -8.64, -7.44, -5.76, -4.88, -1.28, -0.72, 0.88),$$

while the orthogonal-equilibrium spectrum has six -2 modes and three zero modes, up to floating-point error. The positive initial mode is physical growth, not a spurious instability.

`report_v3/manifest.json` records SHA-256 hashes of this report's source, figures, data, verification record and code snapshot. `report_v1/` remains a separate preserved snapshot, with its corresponding project state identified by `report-v1`. Further deliveries should use a new directory such as `report_v3/` or `report_final/` and update the version index.